Introduction to Artificial Intelligence

Chapter 2 - Part I: Agents, Rationality, and PEAS

Namyup Kim

Soongsil University



What Is an Agent?

1
What is an agent?

2
Rationality

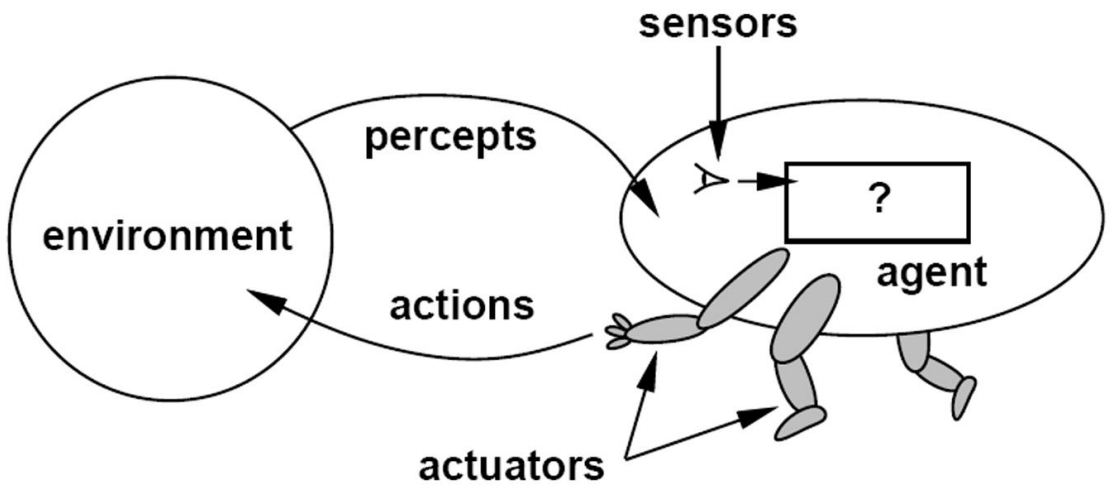
3
PEAS
(Performance, Environment, Actuators, Sensors)

4
Environment types

5
Agent types

What Is an Agent?

- An **agent** perceives its **environment** through **sensors** and **acts** upon it through **actuators**.



- Control theory
  - A **closed-loop system** that **senses** and **corrects** to the desired states without human intervention - the thing doing it is called a **controller (ex, thermostat)**



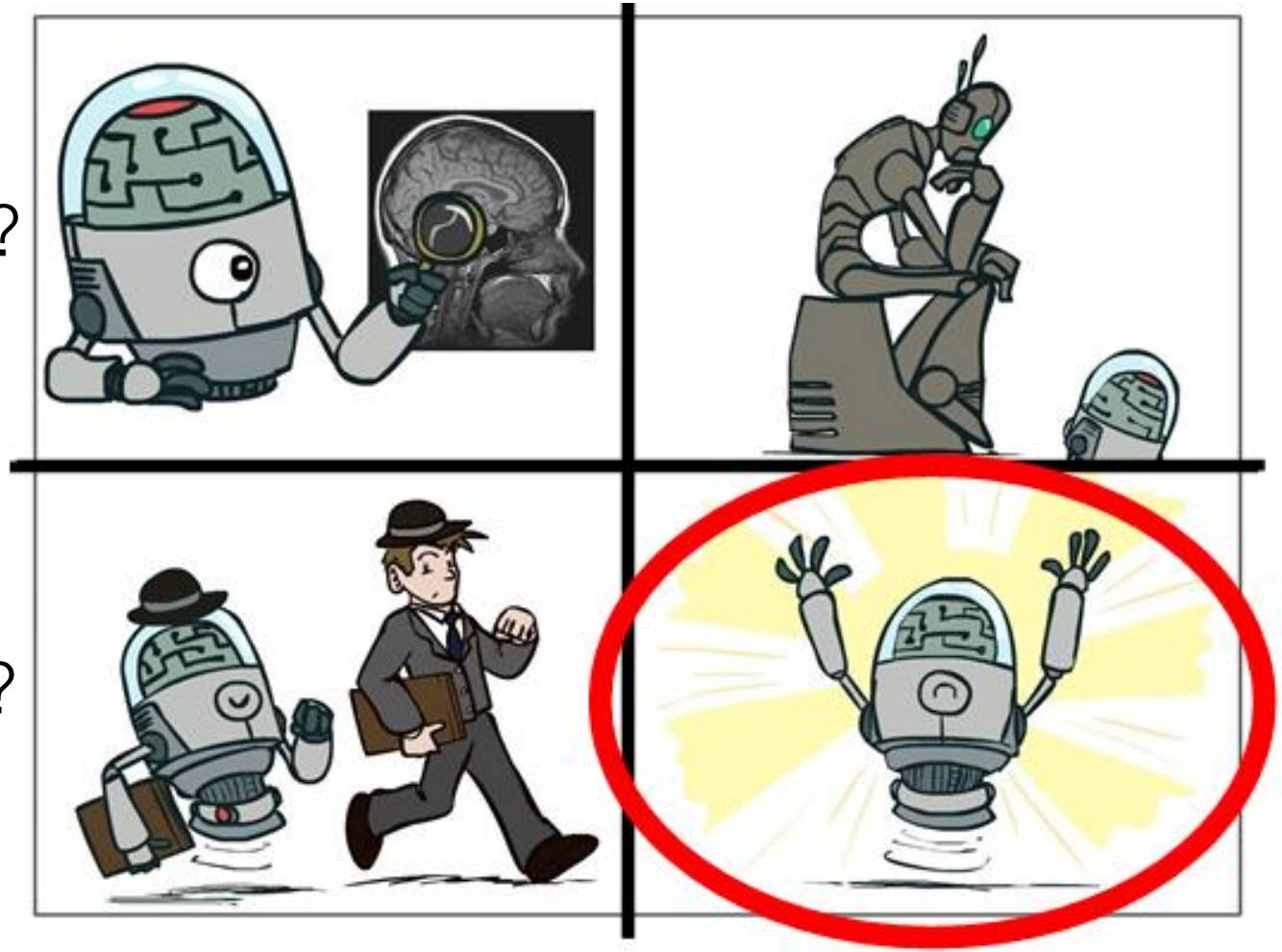
- Softbot, short for software robot
  - An agent whose whole world is inside a machine
  - Its sensors read files and packets; its actuators write them

A Rational-Agent Perspective

Should we make machines that…

Think like people?

Think rationally?

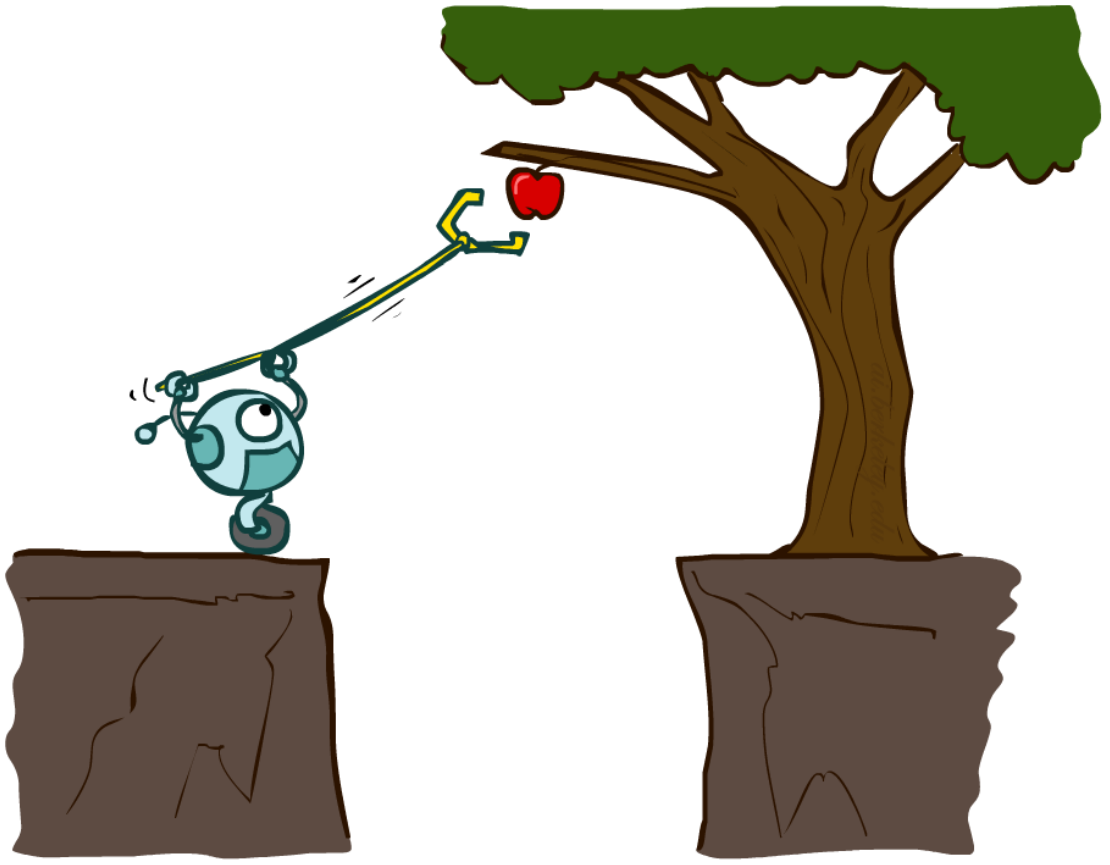


Act like people?

Act rationally?

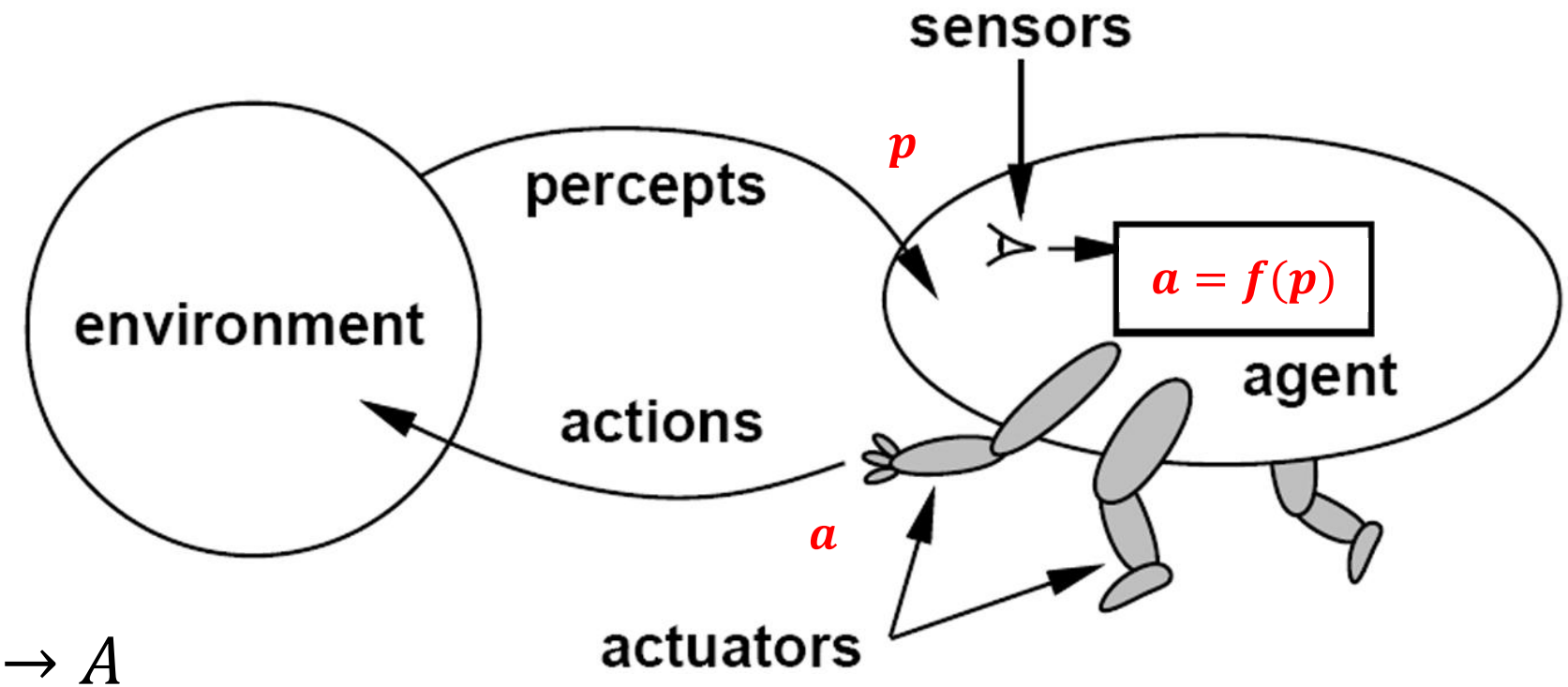
Designing Rational Agents

- An **agent** is an entity that perceives and acts.
- A **rational agent** selects actions that maximize its (expected) utility.
- Characteristics of the **percepts**, **environment**, and **action space** order techniques for selecting rational **actions**



Agent Function and Agent Program

- The ***agent function*** maps percept sequences P^* to a set of actions A.

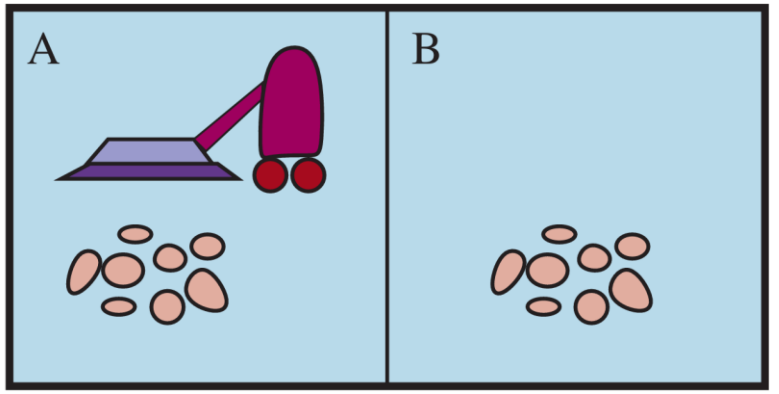


$$f : P^* \rightarrow A$$

- The ***agent program*** is the implementation that produces it

Example: The Vacuum World

- **Sensors**: maybe camera (Computer Vision)
- **Percepts**: [location, status]
  - {[A, Dirty], [A, Clean], [B, Dirty], [B, Clean]}
- Four **actions** are available:
  - {Left, Right, Suck, NoOp}



Agent **function**: $f : P^* \rightarrow A$

Percept sequence	Action
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A, Clean], [A, Clean]	Right
[A, Clean], [A, Dirty]	Suck
…	…

Problem: This table can become infinitively large!

Implemented agent **program**:

Most recent percept p

function Vacuum-Agent([location, status])
returns an action a

```
if status = Dirty then return Suck
else if location = A then return Right
else if location = B then return Left
```

Rationality

1
What is an agent?

2
Rationality

3
PEAS
(Performance, Environment, Actuators, Sensors)

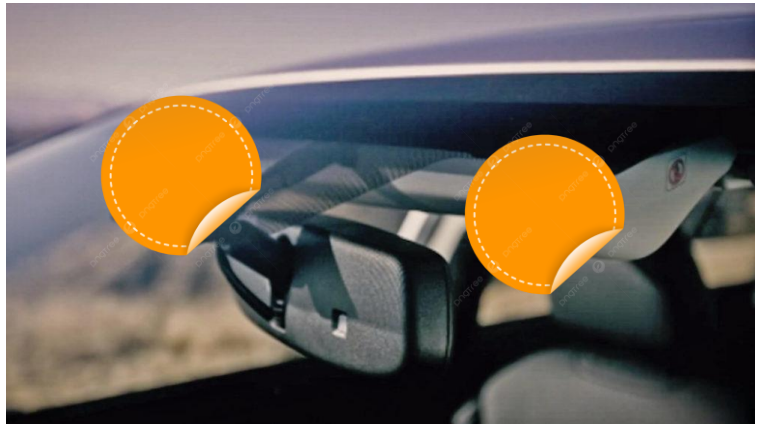
4
Environment types

5
Agent types

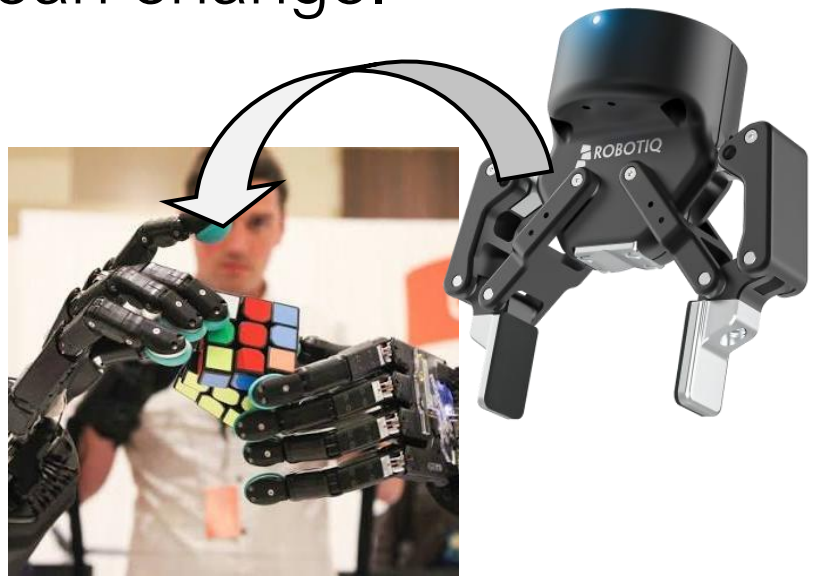
Rational Agents: What Is the Right Action?

- Whether an agent is rational depends on four things:
  1. The **performance measure** that defines the criterion of success
  2. The agent's **prior knowledge** of the environment
  3. The **actions** that the agent can perform
  4. The agent's **percept sequence** as history

- Change any one of the four, and the right action can change.







The Definition of a Rational Agent

- Definition: For every possible percept sequence (history), a rational agent s elects the action **expected to *maximize its performance measure***.
- **Performance measure**:
  - The objective criterion for success
  - Scores the sequence of environment states the agent brings about
  - Plays the role of a utility function, or a reward function
- **Expectation**:
  - The average over all outcomes that could occur, each weighted by how likely it is
- **Rule**:
  - Among the available actions a, take the one whose expectation is highest

$$a = argmax_{a \in A} E(U|a) \quad \text{where} \quad E(U|a) = \Sigma_{\text{outcome}} U_a \times P$$

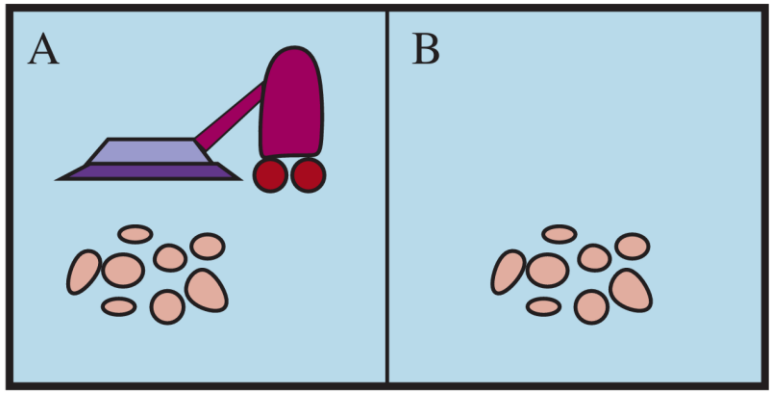
What Rationality Is Not

- Rationality is an ideal. A standard we design toward, not a description of any real system
  - **Not omniscience**: It cannot know how things will actually turn out
    - You cross a street. Nothing is coming. A cargo door falls from a passing aircraft and lands on you
      - Were you irrational to cross? **No.**
  - **Not perfection**
    - Rationality maximizes **expected** performance;
    - Perfection maximizes actual performance
  - **Not passive**
    - It must gather information, learn from what it perceives, and explore what it does not yet know
  - **Not unlimited**
    - Bounded by available memory, computation, and sensors

Example: Is the Vacuum Agent Rational?

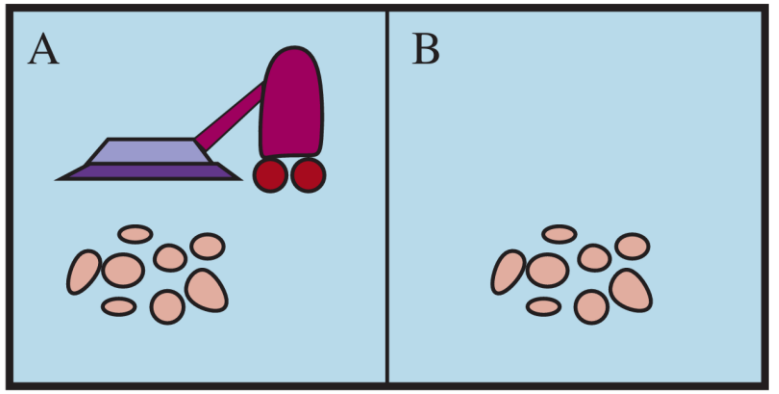
- Two questions worth asking of our vacuum agent:
  - What should the performance measure be?
  - Given that measure, is this agent rational?
- The honest answer to the second is: it depends
- And what it depends on is exactly the 4 things from earlier

  performance measure, prior knowledge , actions, percept sequence



Example: Is the Vacuum Agent Rational?

- Two questions worth asking of our vacuum agent:
  - What should the performance measure be?
  - Given that measure, is this agent rational?
- The honest answer to the second is: it depends
- And what it depends on is exactly the 4 things from earlier



Agent function: $f : P^* \rightarrow A$

Percept sequence	Action
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A, Clean], [A, Clean]	Right
[A, Clean], [A, Dirty]	Suck
…	…

Problem: This table can become infinitively large!

Implemented agent program:

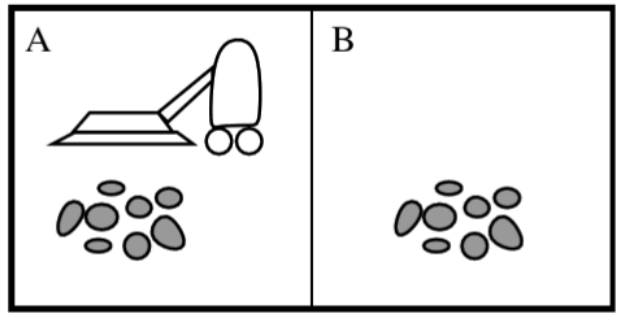
Most recent percept p

function Vacuum-Agent([location, status])
returns an action a

```
if status = Dirty then return Suck
else if location = A then return Right
else if location = B then return Left
```

Pin Down the Assumptions

- **Performance measure**
  - One point per clean square, per time step, over 1000 steps
- **Prior knowledge**
  - The layout is known. Where the dirt is, and where the agent starts, are not
- **Four actions are available:**
  - {Left, Right, Suck, NoOp}
- **Percepts**: [location, status]
  - {[A, Dirty], [A, Clean], [B, Dirty], [B, Clean]}



Implemented agent program:

function Vacuum-Agent([location, status])
returns an action a

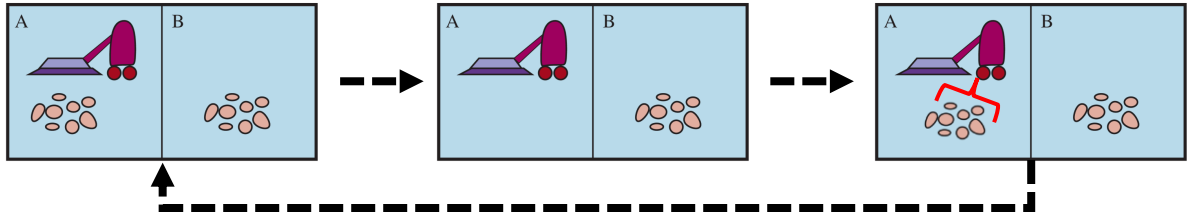
```
if status = Dirty then return Suck
else if location = A then return Right
else if location = B then return Left
```

Under Those Assumptions: Yes

- This simple agent is rational
- No other agent does better, because there is nothing better to do
  - Square dirty? Cleaning it is the only way to score
  - Square clean? The only remaining points are next door
- A three-line program turns out to be optimal
- Which should make you suspicious of the assumptions
- Rationality is a relation between **an agent and its environment**, not a property the agent acts

Design the Measure Around the Outcome

- Suppose we score the vacuum on dirt collected per shift (performance me asure)
- A sufficiently good agent will maximize exactly that
  - Clean the floor. Throw it back out. Clean it again, repeatly



  - It is doing what we asked. That is the problem
- The rule:
  - **The measure around what you want the world to look like;** not how you imagine the a gent have to behave.
- Same lesson as last week - the machine optimizes what we wrote down..
  → Hallucination in a LLM is occurred 😢 .

PEAS

1
What is an agent?

2
Rationality

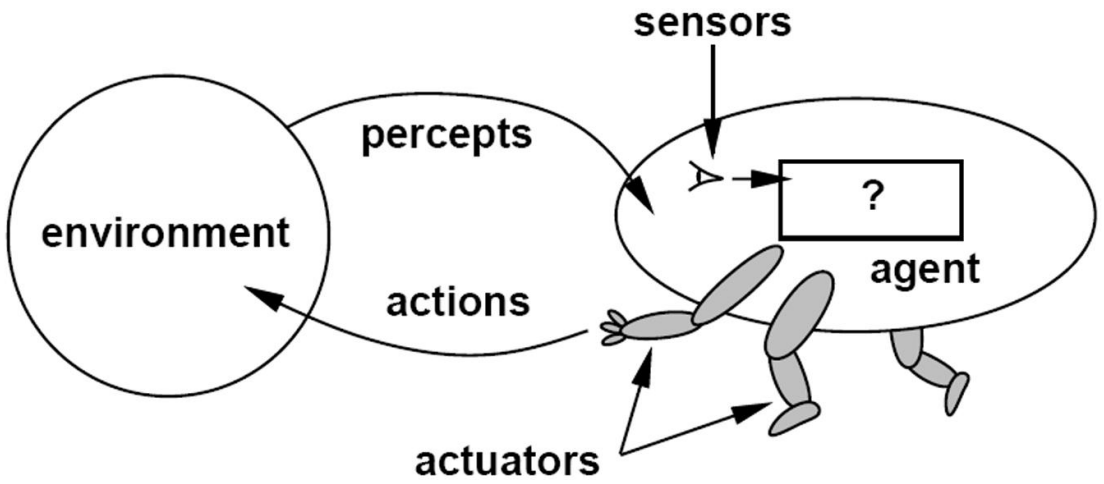
3
PEAS
(Performance, Environment, Actuators, Sensors)

4
Environment types

5
Agent types

Problem Specification: PEAS

- Rationality depends on four things.
  - That bundle (PEAS) is the ***Task environment.***
  - In designing an agent, the first step must always be to ***specify the task environment*** as fully as possible.



Performance measure	Environment	Actuators	Sensors
Winning or losing a game. Stability and safety while driving	The chess board. The road, the weather, the other traffic	A robot arm. The steering, the accelerator, the brake	A camera, a lidar, a microphone
The criterion of success	Everything outside that the agent deals with	the ways of affecting the environment	the ways of observing it

Example: An Automated Taxi Driver

Performance measure	Environment	Actuators	Sensors
Safe, fast, legal Comfortable trip Profitable Easy on other road users	Roads and traffic Pedestrians, police, Customers, Weather	Steering, accelerator Brake, signals, horn Display and speech	Cameras, radar, lidar Speedometer, GPS Engine sensors Accelerometer, microphone

- "Roads" covers a farm track and a twelve-lane motorway
- Snow in Gangwon. No snow in Busan
- Drive on the right here. On the left in Japan
- Every restriction you can honestly make shrinks the design problem
  - A taxi for one campus is a different machine from a taxi for one country
- Notice the performance measures conflict: fast fights safe, profitable fights comfortable. Stating the ***conflict*** is part of the specification.

PEAS for Other Agent Types

Agent	Performance	Environment	Actuators	Sensors
Medical diagnosis	Patient health, cost	Patient, hospital, staff	Questions, tests, treatments	Symptom and finding entry
Satellite image analysis	Correct categorization	Satellite, downlink, weather	Category display	High-resolution camera
Part-picking robot	Parts in the right bin	Conveyor belt, bins	Jointed arm and hand	Camera, joint angle, touch
English tutor	Student's later test score	Students, testing agency	Exercises, feedback, speech	Keyboard, voice

Let’s specify a task environment of a rational agent

- Write the PEAS description for one of these:
  - A robot vacuum in a real apartment
  - A system that recommends what to watch next
  - A drone that inspects bridges for cracks
  - An agent that plays a game you know well
- Four columns. Be specific - “the internet” is not an environment
- The hardest column is almost always P. Try to make it gameable, then fix it.

Introduction to Artificial Intelligence

Chapter 2 - Part II: Environment and Agent types

Namyup Kim

Soongsil University



Environment types

1
What is an agent?

2
Rationality

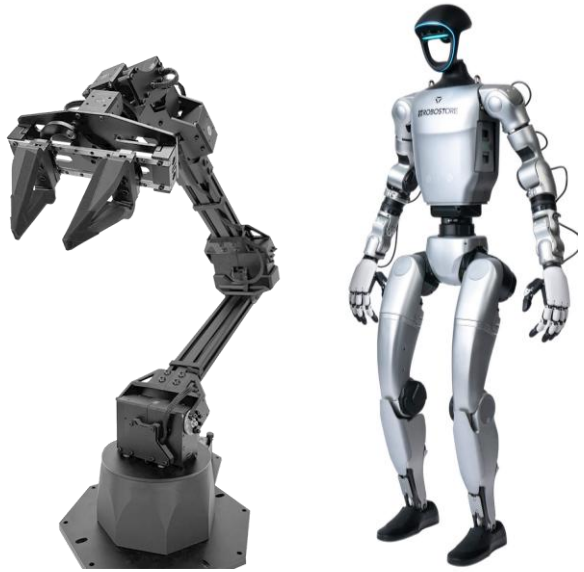
3
PEAS
(Performance, Environment, Actuators, Sensors)

4
Environment types

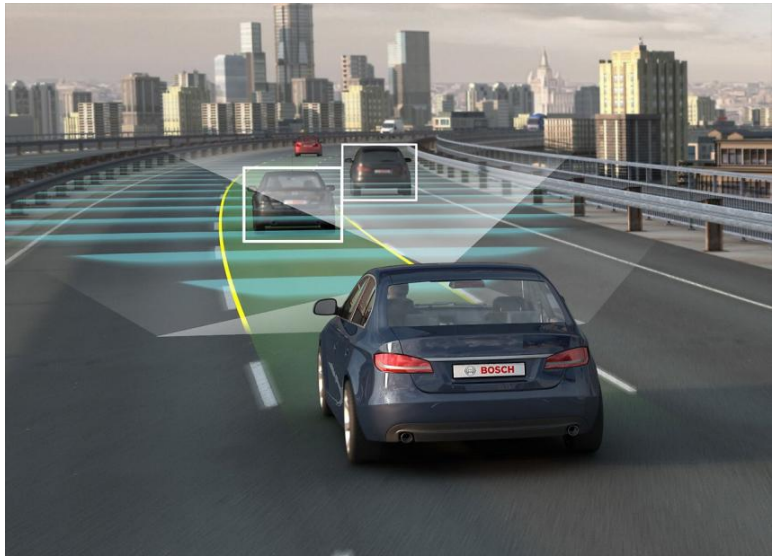
5
Agent types

What Counts as the Environment?

- The environment is everything except the agent function
- This leads somewhere strange:
  - The **sensors** and **actuators** are part of the **environment** too.. 🤔
    - Ex: 1080p vs 720p, High-torque actuator..
  - The agent function is only the mapping from percepts to actions
  - The camera that produces the percept is on the outside of that line



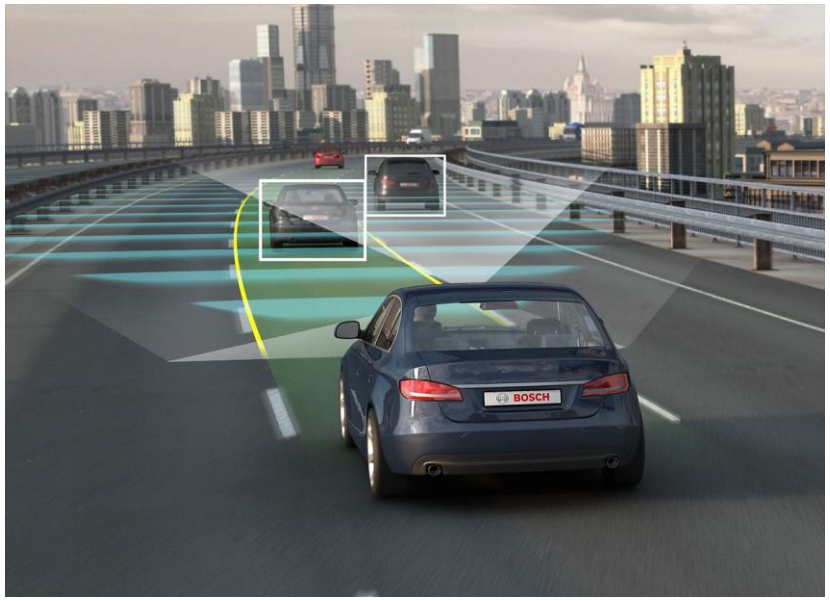




Environment Types: Fully or Partially Observable

- Fully observable - the sensors deliver everything relevant to the decision, at every step
- Partially observable - something relevant is hidden
  - Noisy or inaccurate sensors
  - Or state that simply is not sensed





Environment Types: Single-Agent or Multiagent

- The test is there another entity whose success depends on what you do?





- Careful with the test - a falling rock is not an agent
  - It is only an agent if modelling it as maximizing something is the useful thing to do

Environment Types: Competitive and Cooperative

- Multiagent settings come in two flavors, and usually both at once
- Competitive
  - Chess: your opponent's performance measure is the opposite of yours
- Cooperative
  - Taxi-driving environment: avoiding collisions maximizes the performance measure of all agents.
- Driving is partly both
  - Only one car fits the parking space: as fast as possible, avoiding collision.
- Multiagent settings make two odd things rational:
  - communicating, and behaving unpredictably - Poker



Environment Types: Deterministic, Nondeterministic, Stochastic

- **Deterministic** - the next state of the environment is completely determined by the current state and the action executed by the agent(s).
- **Nondeterministic** - several outcomes are possible, and we do not put numb ers on them
  - Ex) “It might rain”
- **Stochastic** - the model attaches probabilities
  - Ex) “A 25% chance of rain”
- One caution: If the environment is partially observable, however, then it co uld appear to be nondeterministic.
  - The world is not rolling dice - you just cannot see the whole board

Environment Types: Episodic or Sequential

- Episodic - experience comes in independent pieces
  - Perceive, act, done. The next episode does not care what you did
  - Ex) Figuring broken parts on a production line

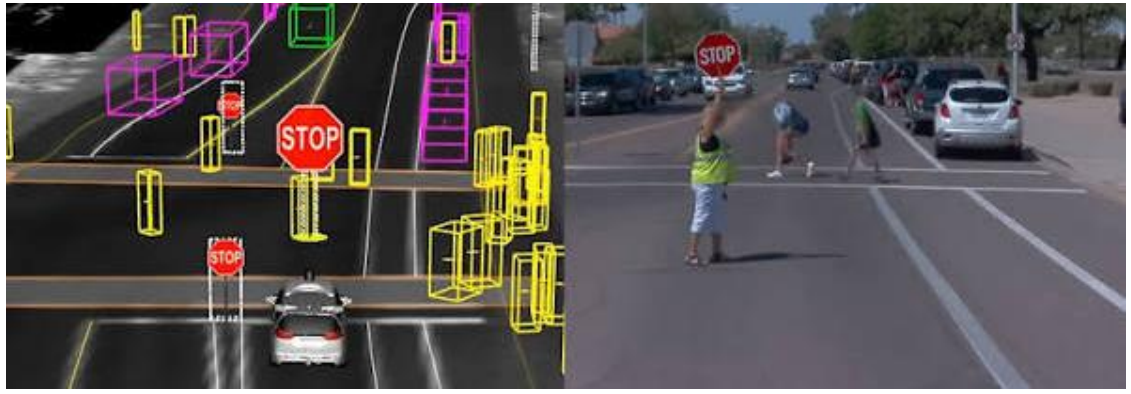
- Sequential - this decision shapes every decision after it
  - Ex) Chess. Driving. Almost everything interesting



Environment Types: Static, Dynamic, Semidynamic

- Static - the world waits while the agent thinks
  - Ex) A crossword puzzle does not move, thinking (waiting) is free.
- Dynamic - the world changes during consideration
  - Ex) Driving. And note: not deciding (waiting) is itself a decision.
- Semidynamic - the world holds still, but your score does not
  - Chess with a clock
- Dynamic environments punish slow reasoning, which constrains which alg orithms you can use at all.





Environment Types: Discrete or Continuous

- This applies to three things at once: **state**, **time**, and **percepts/actions**
- Chess/Go: finitely many states, distinct moves, discrete throughout
- Driving: position and speed sweep smoothly, steering angle is continuous, t ime does not tick

Environment Types: Known or Unknown

- This one is not a property of the environment at all
- It is about the agent's or designer's knowledge of the rules





- The ***performance measure*** itself can be unknown
  - Ex) A taxi does not know whether this passenger wants speed or calm
  - Ex) Which quietly turns a single-agent problem into something closer to multiagent

Let's Think About It

- For each one: observable, agents, deterministic, episodic, static, discrete?
  - Crossword puzzle
  - Chess with a clock
  - Poker
  - Backgammon
  - Taxi driving
  - Medical diagnosis

Let's Think About It — Answers

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

Figure 2.6 Examples of task environments and their characteristics.

Not always clear cut. Medical diagnosis is single-agent only if you decline to model the patient as one, and episodic only if the task is one diagnosis rather than a course of treatment.

The Environment Decides the Design

- This is the payoff of the whole section. Each property makes a demand:
  - Partially observable → the agent needs memory
  - Nondeterministic → it must prepare for possibility
  - Multiagent → acting unpredictably may become rational
  - Static → there is time to compute a good decision
  - Continuous time → it becomes a controller, always running
  - Unknown rules → it must explore
  - Unknown performance measure → it must learn from the person it serves

Agent types

1
What is an agent?

2
Rationality

3
PEAS
(Performance, Environment, Actuators, Sensors)

4
Environment types

5
Agent types

Why Not Just Use a Table?

- We said any agent function can be written as a table
  - So why not build the table and look the answer up?
  - Chess would need somewhere around 10^150 entries. Too huge..

Percept sequence	Action
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A, Clean], [A, Clean]	Right
[A, Clean], [A, Dirty]	Suck
…	…

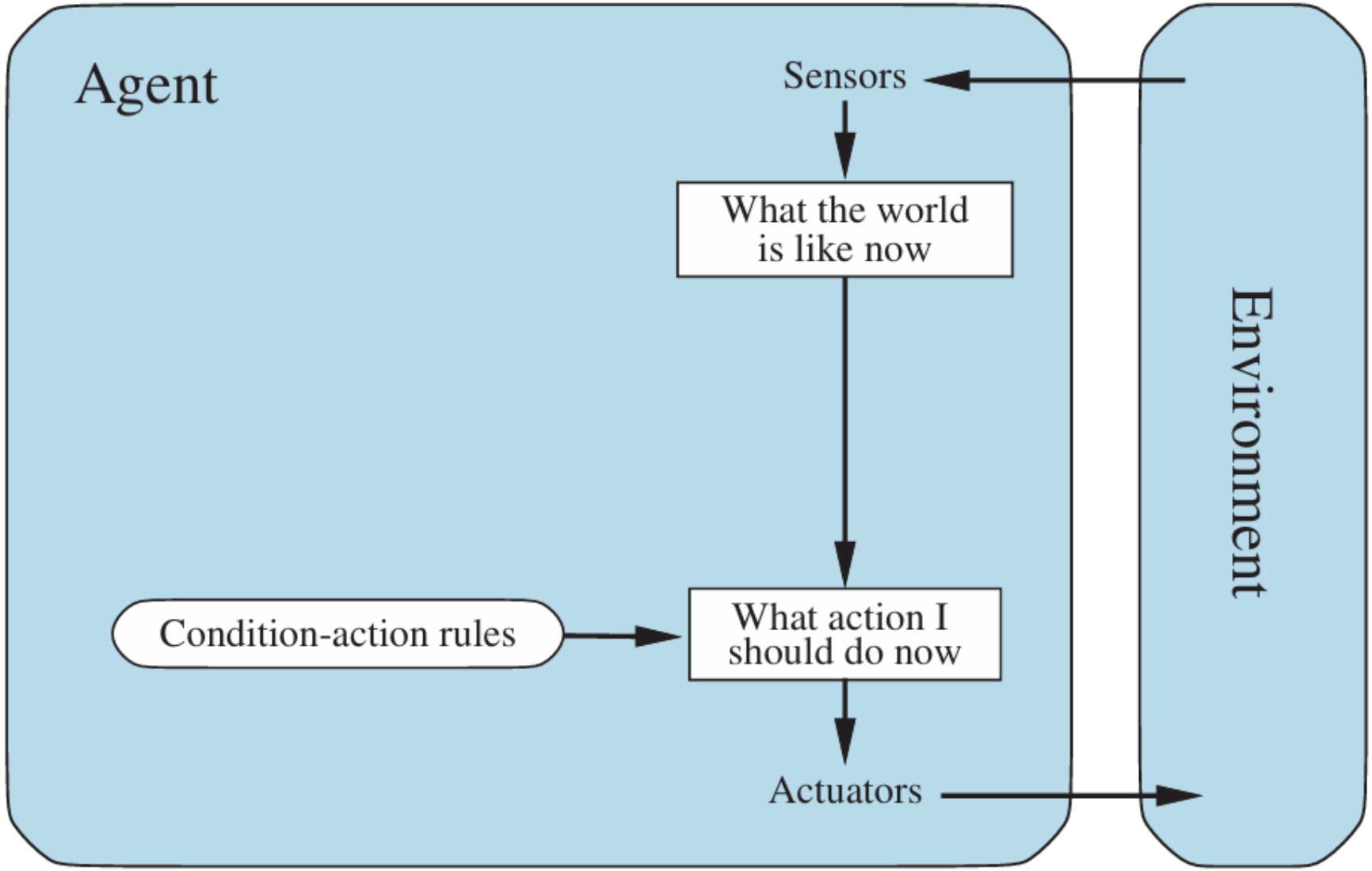
- Three separate walls, any one of them fatal:
  - No machine could store it
  - No designer could fill it in
  - No agent could learn all the right entries

- So the rest of the chapter asks: **how do we get the same behavior out of a small program?**

A Hierarchy of Agent Types



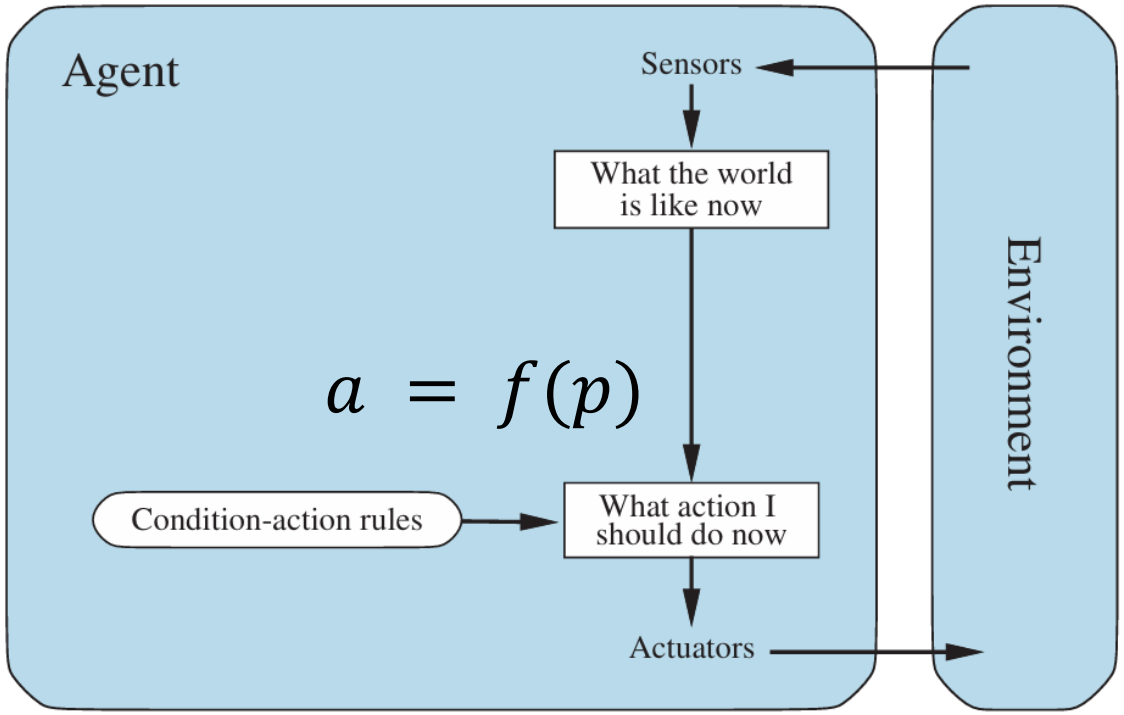
Simple Reflex Agent



Acts on the current percept alone. If the car in front is braking, brake. Everything before this instant is ignored.

How a Reflex Agent Runs

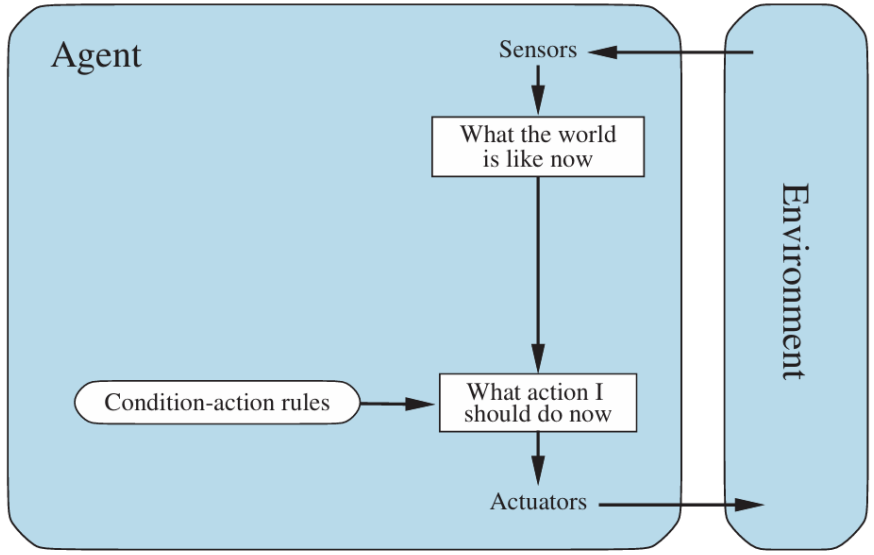
- The agent function is the current percept alone: $a = f(p)$
- So, the interaction is a simple alternation: $p_0, a_0, p_1, a_1, p_2, a_2, \ldots p_t, a_t$
- Fast, and cheap. It also does not know why it does anything
  - It does not know the performance measure at all - yet well-chosen rules can still p erform well



Example of Simple Reflex Agent

- Rule-based reflex vacuum cleaner:

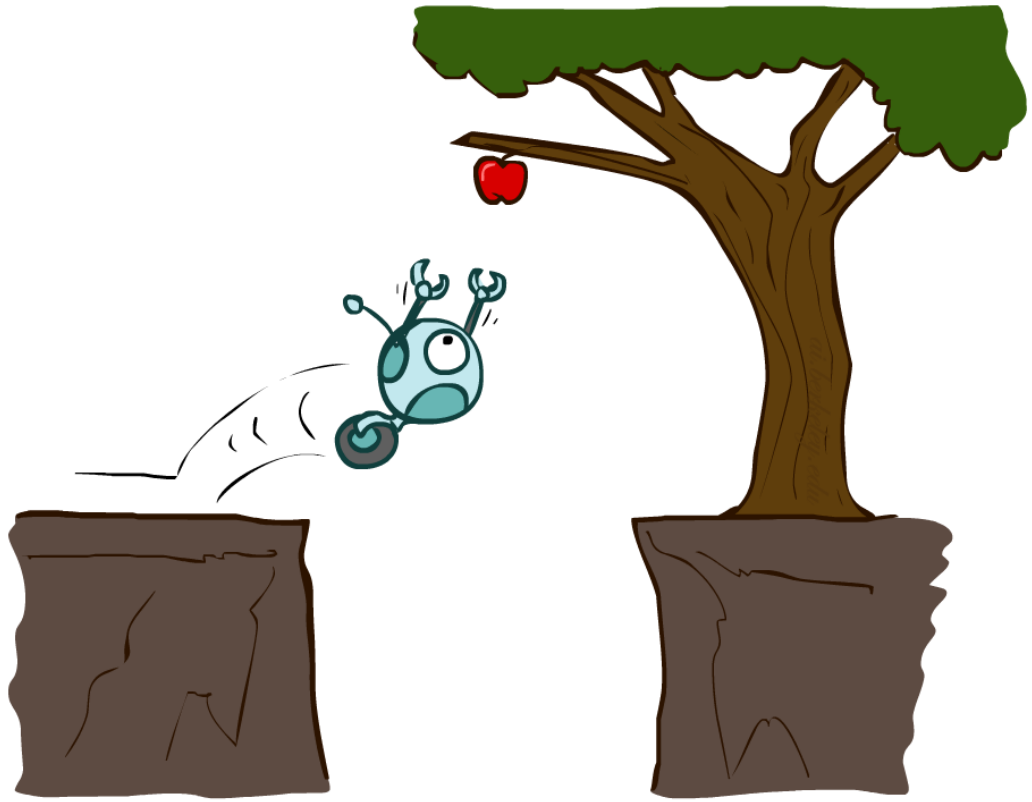
```
function REFLEX-VACUUM-AGENT([location,status]) returns an action
    if status = Dirty then return Suck
    else if location = A then return Right
    else if location = B then return Left
```



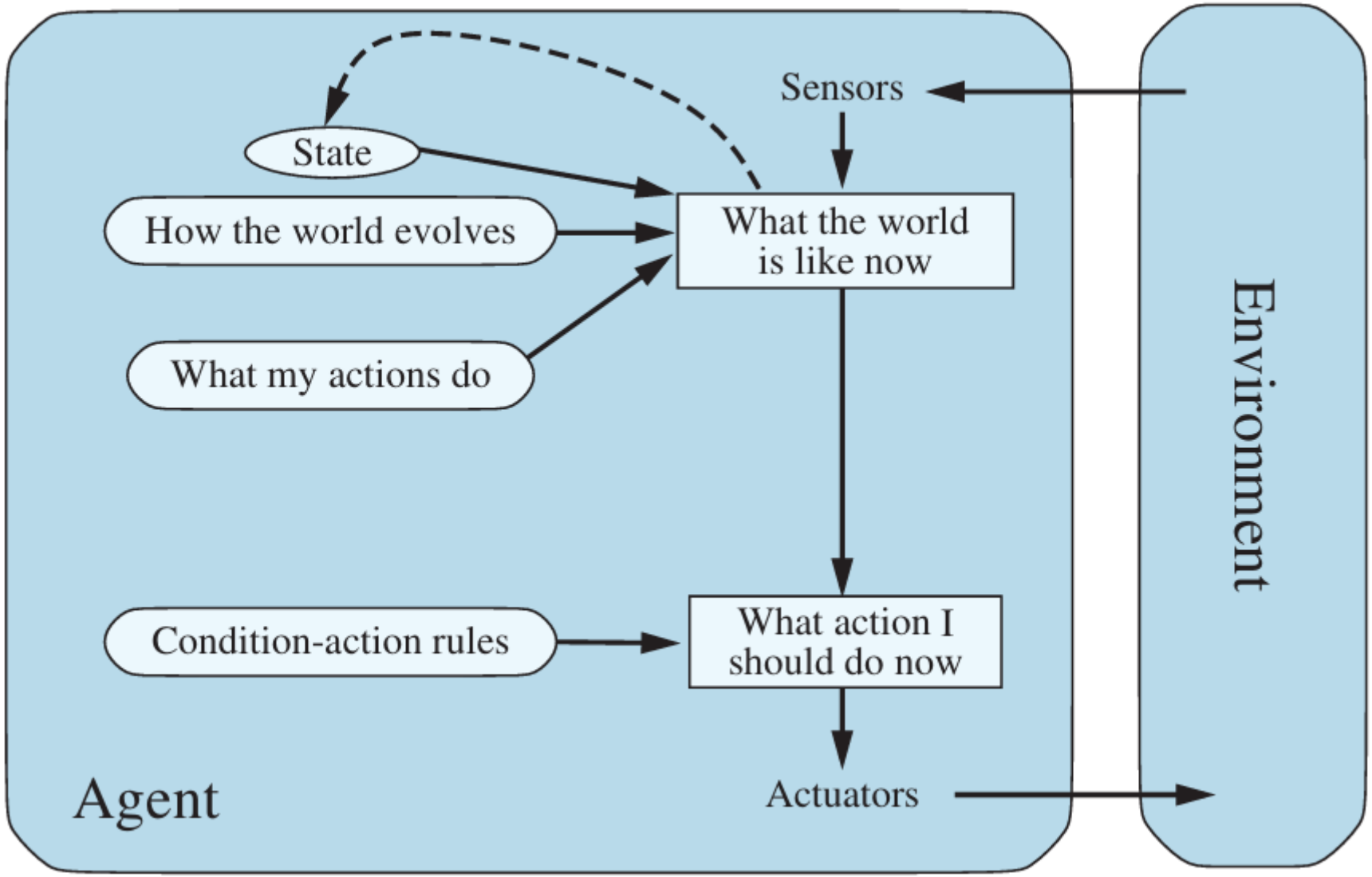
- Driving: If the car in front is braking, brake. Everything before this instant is ignored.

Where Simple Reflex Breaks

- Take the **location** sensor away. Now it knows only clean or dirty
- On [Clean], what should it do? It cannot tell A from B
  - Always Left: stuck forever if it started in B
  - Always Right: stuck forever if it started in A
- A little unobservability is enough to trap it in a loop
- One escape: ***randomize***
  - A coin flip reaches the other square in two steps on average
  - A useful trick, not a real fix. Better is to give it a memory



Model-Based Reflex Agent

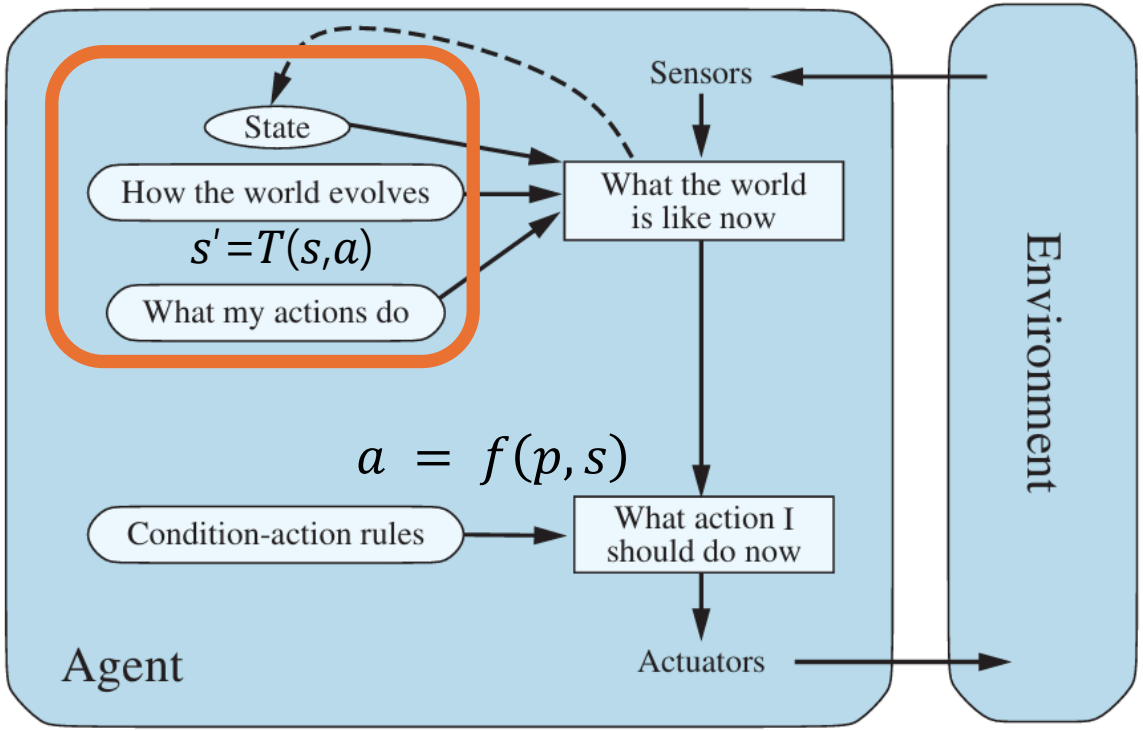


Model-Based Reflex Agent: Adding Memory

- The agent function now takes a second argument
  : $a = f(p, s)$ where s is state, updated at every step.

- It can now track what it cannot currently see
  - Memorize a car in the next lane, while you are looking the other way
  - Memorize which rooms already cleaned

- More information available means better rules become possible

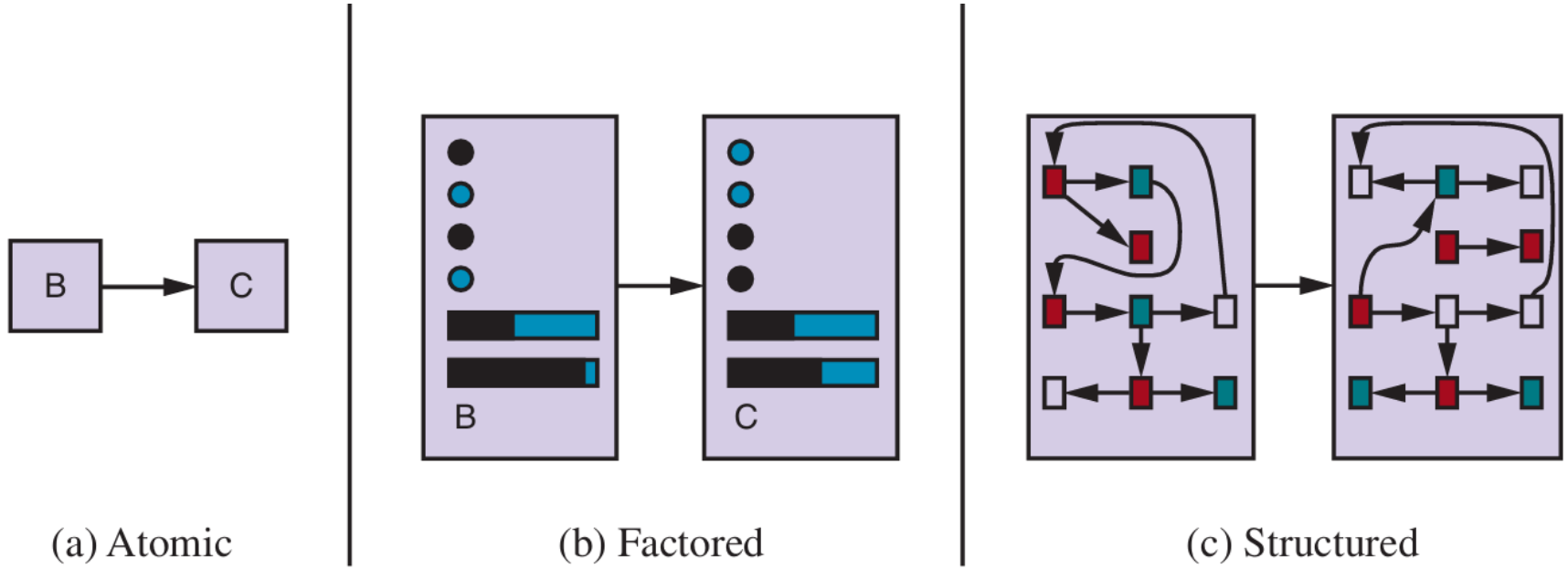


Model-Based Reflex Agent: Two Models Are Needed

- To maintain that **state variable**, the agent needs two kinds of knowledge
- A **transition model**:
  - How the world changes - both from its own actions and on its own
- A sensor model
  - How the state of the world shows up in its percepts
- One honest caveat: the state is a best guess in by sensor, never a certainty
  - In a partially observable world the exact state is rarely knowable

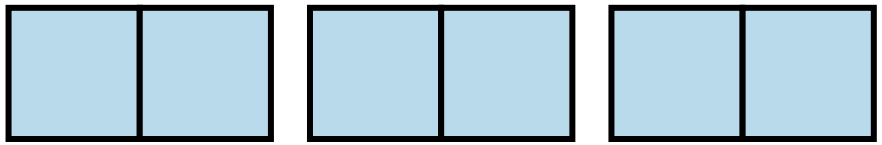
Model-Based Reflex Agent: State Representation

- State helps the agent keep track of the environment
  - Atomic - the state is a label.
    - Just “A” or “B”.
  - Factored - the state is a set of attribute-value pairs, called fluent
    - [location = A, status = clean, temperature = 24°C]
  - Structured - objects, with attributes and relations between them
    - “the cup is on the table, which is in the kitchen”



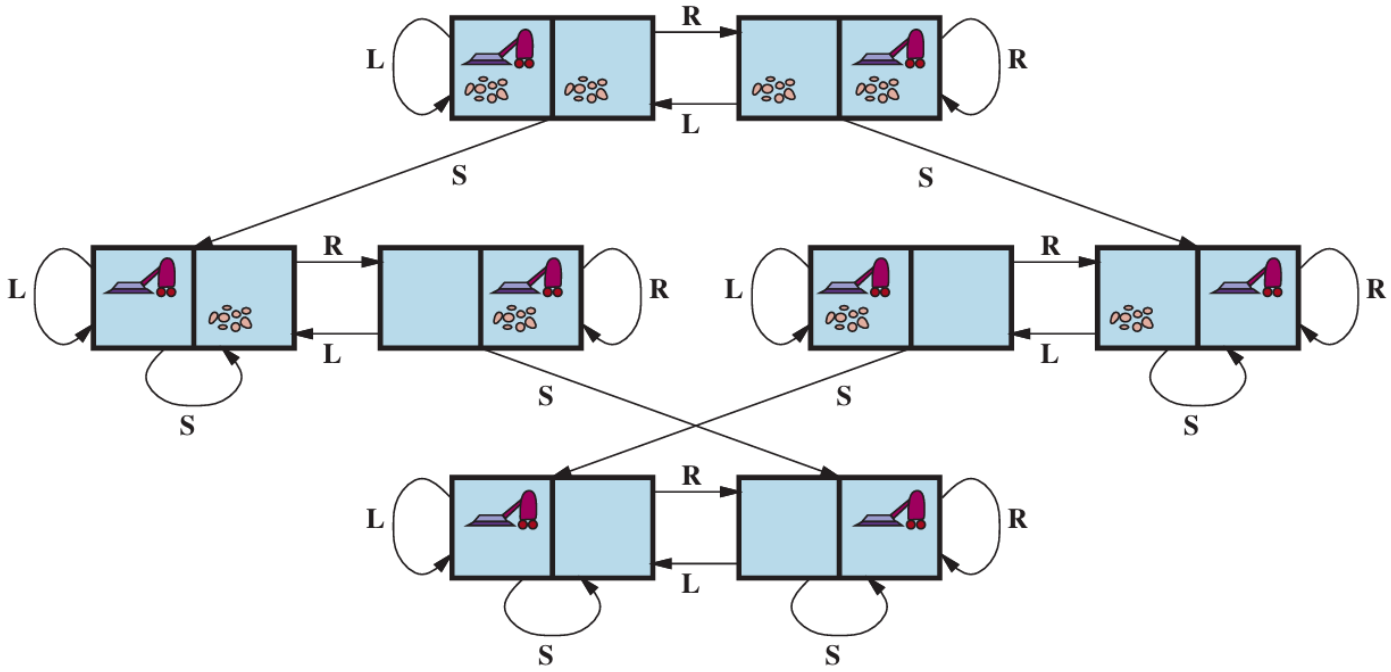
Model-Based Reflex Agent: The Transition Model

- Treat the environment as a discrete dynamic system
- The transition function says where an action takes you:
  - the set of states:



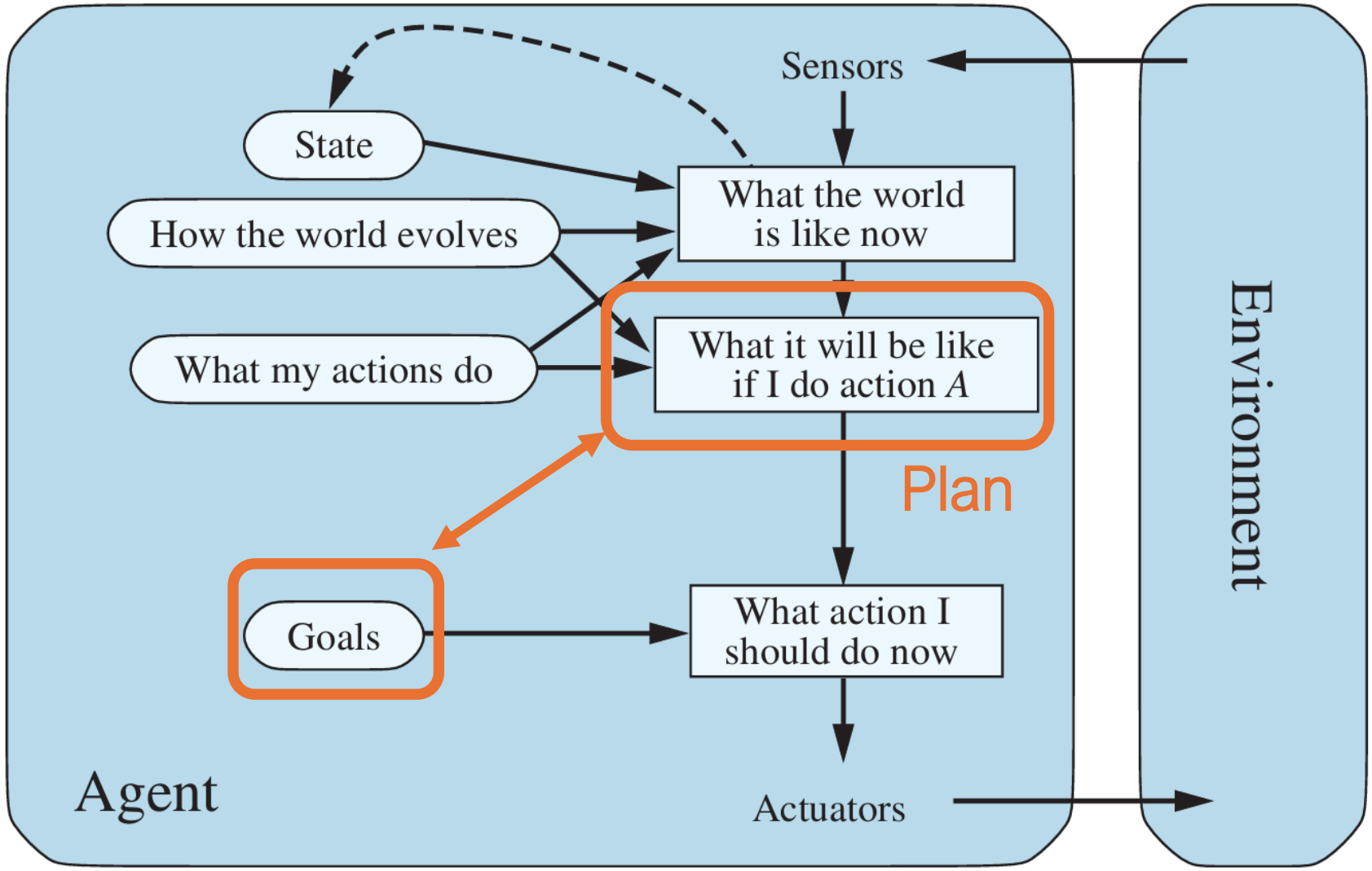
  - A the set of actions:





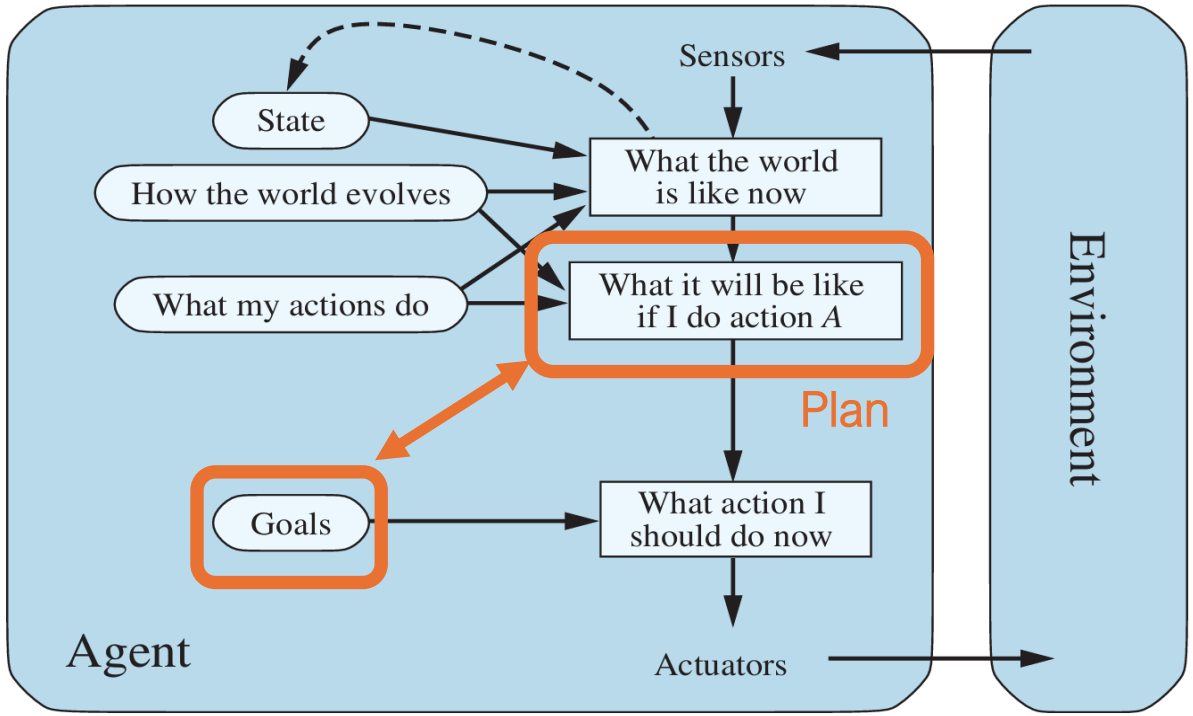
- *Next week this becomes the backbone of every search algorithm*

Goal-Based Agent



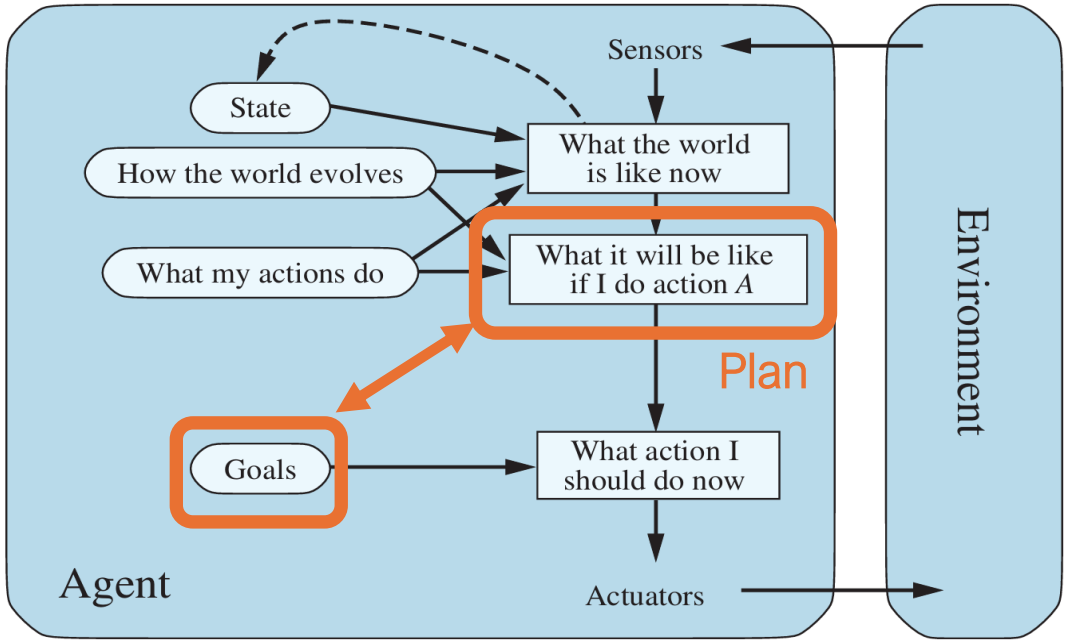
Goal-Based Agent: Flexibility, Concretely

- Has a goal state.
  : The agent is given a defined goal state to reach, and the task **ends** once it gets there.
- Must act toward the goal state.
  : **A planning** uses **search algorithms** to work out a sequence of actions that leads to the goal.

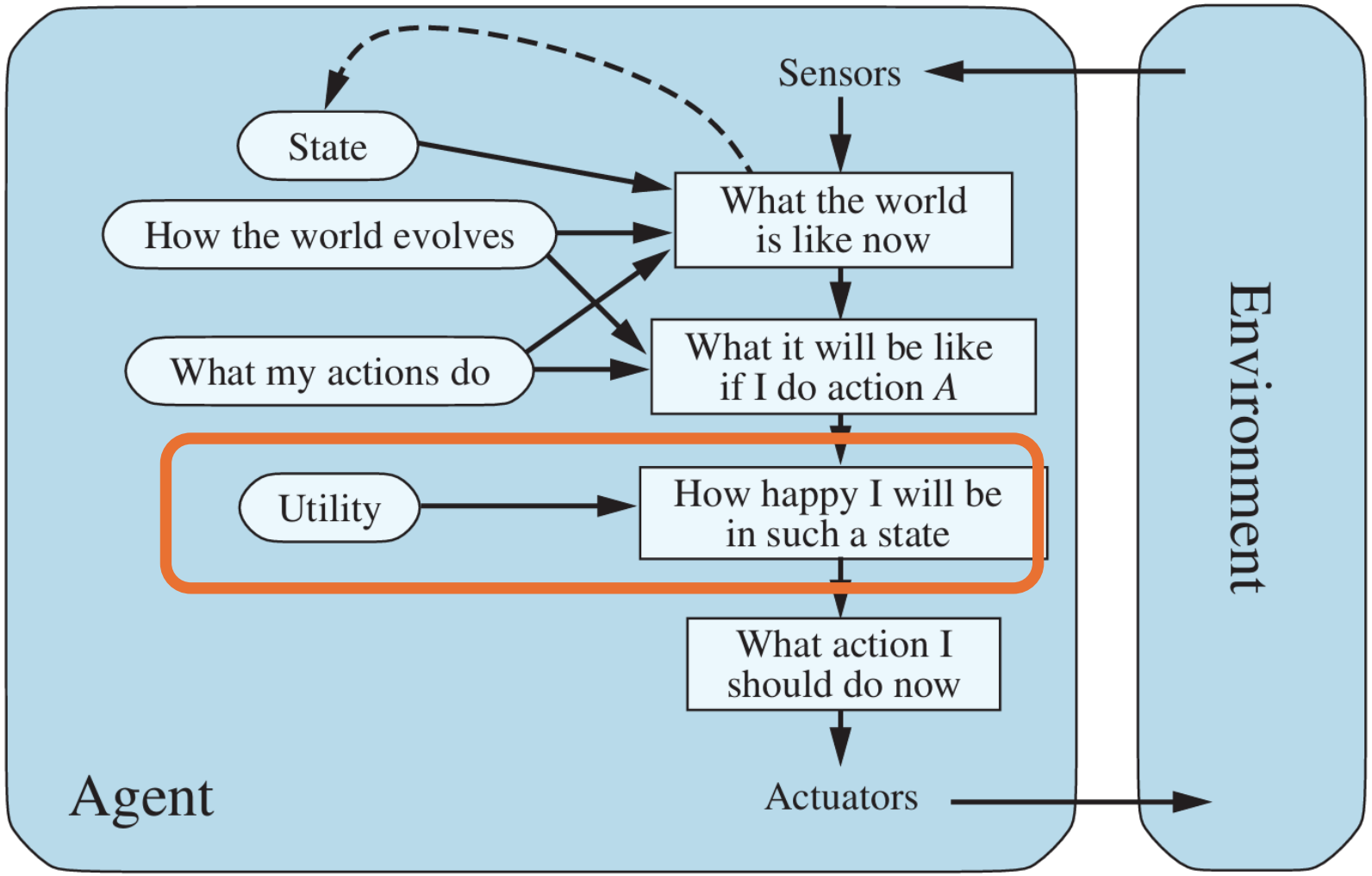


Goal-Based Agent: Examples

- Send a goal-based taxi somewhere new
  - Change one thing: the destination
- Send a reflex taxi somewhere new
  - Every rules for when to turn and when to go straight will work only for a single destination.
  - Replace all of them
- The difference is that the goal agent's knowledge is written down explicitly, and so can be changed



Utility-Based Agent



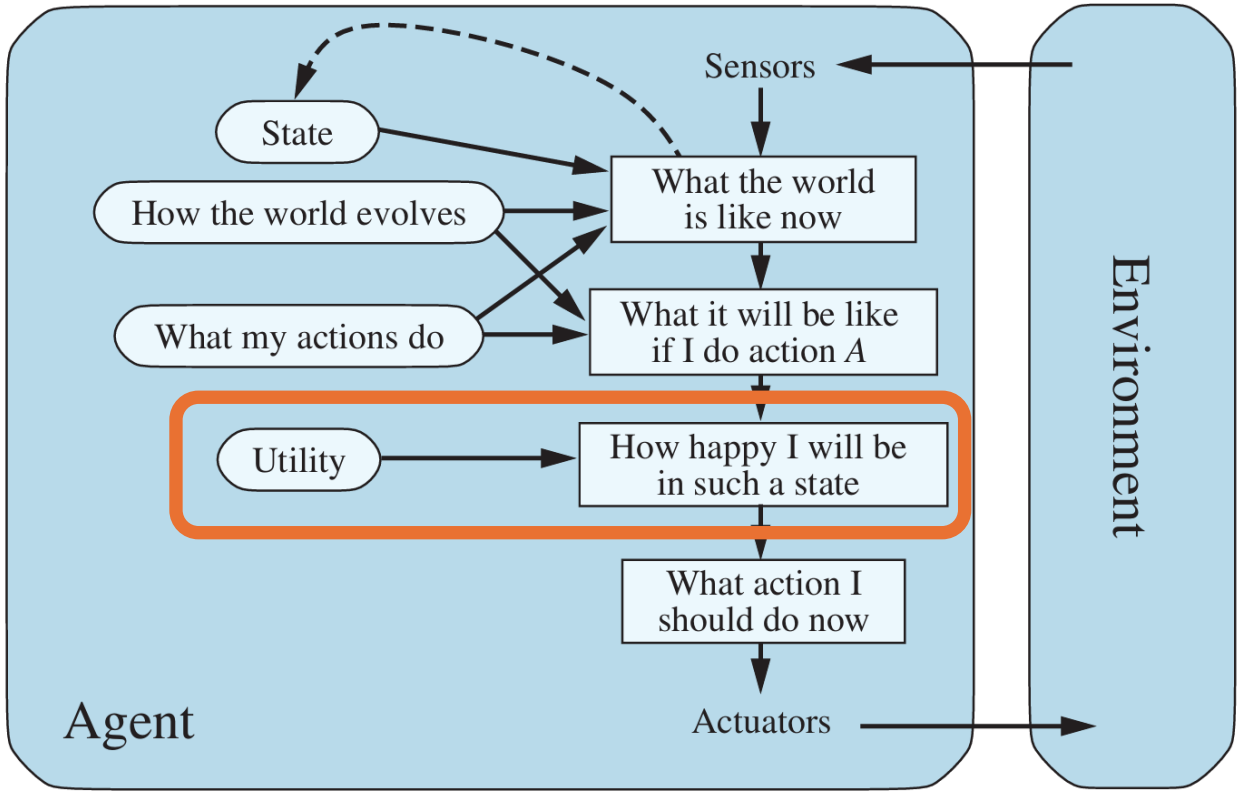
A goal is a yes-or-no test. A utility function scores how good a state is, which is what you need when several routes reach the goal.

Utility-Based Agent: When Goals Are Not Enough

- Three situations a goal test and a path cost cannot handle
  - There is no terminal goal at all
    - A Mars rover keeping its battery out of the danger zone. Nothing to arrive at - the task never ends
  - Objectives conflict
    - Fast against safe against comfortable. You cannot have all three, and someone has to write down the trade-off
  - Outcomes are uncertain
    - 90% chance of a small win, 10% chance of a large one. A yes-or-no test cannot multiply by a pro bability

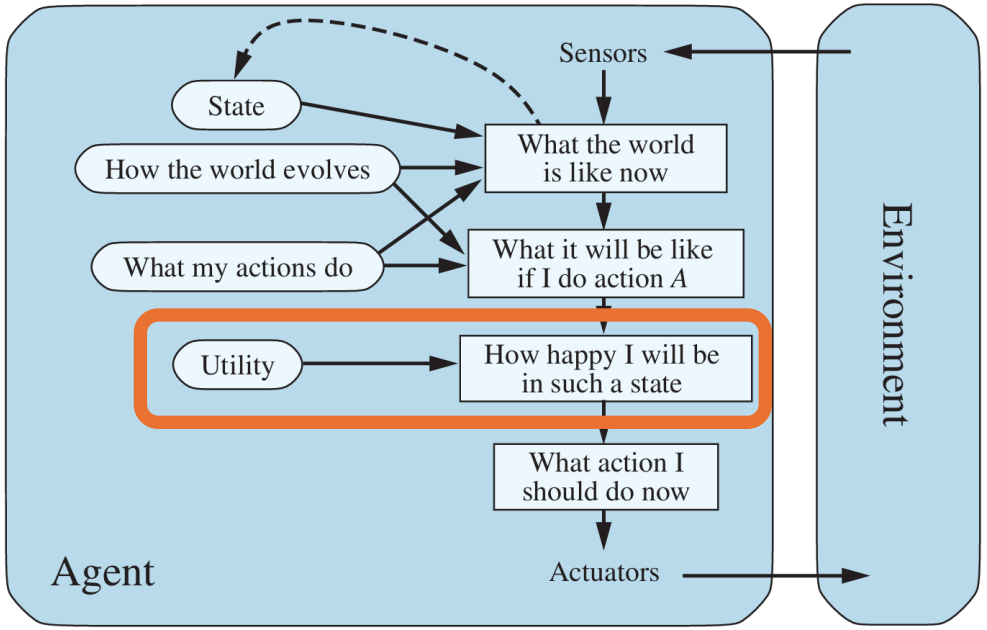
Utility-Based Agent: When Goals Are Not Enough

- Goals just provide a crude binary distinction between “happy” and “unhappy” states.
- A utility function scores how desirable (happy) a state is
  - Often written as a reward $\boldsymbol{R(s)}$ attached to each state.



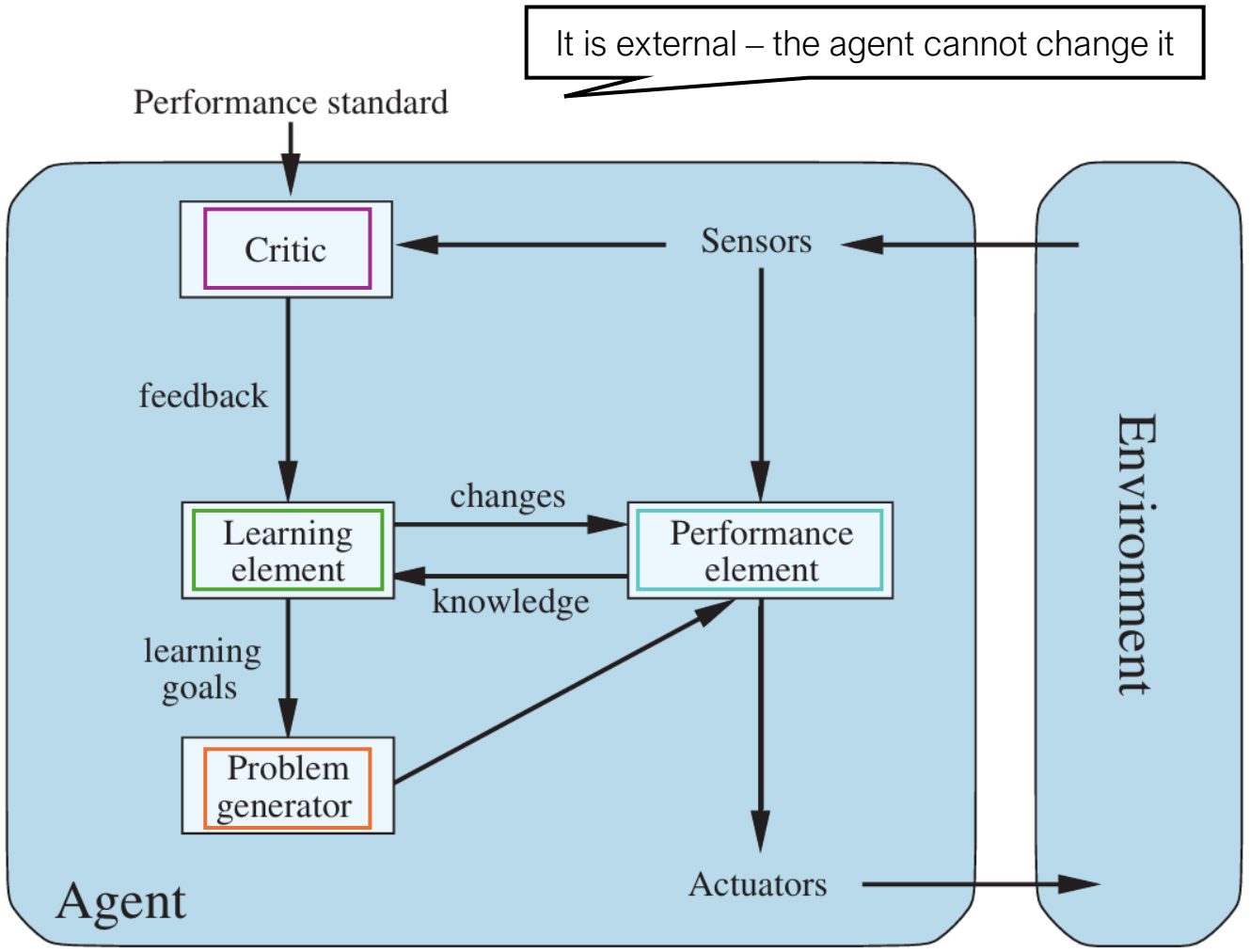
Goal-Based Agent vs Utility-Based Agent

- Goal-based
  - p_0, s_0, a_0, p_1, s_1, a_1, …, s_goal
  - It terminates. The agent arrives, and it is done
- Utility-based
  - p_0, s_0, a_0, p_1, s_1, a_1, …
  - It does not terminate. There is no arriving, only continuing



Learning Agent

- Performance element
  - It takes percepts and chooses actions. That is the entire agent.
- Learning element
  - Updates the performance element so it does better next time
- Critic
  - Judges what happened against a fixed external standard
- Problem generator
  - Suggests intentionally actions worth trying



Learning Agent: Two Things Worth Noticing

- The performance standard must sit outside the agent
  - An agent able to edit its own standard would lower it to whatever it already achieves
  - The same failure as a gameable performance measure, one floor down
- Learning is orthogonal to everything else in this section
  - Any of the four architectures can be built to learn, or not
- And it matters most in unknown environments - which is to say, whenever t he designer is not omniscient

Summary

- An agent perceives through sensors and acts through actuators; the agent function maps percept sequences to actions
- A rational agent takes the action expected to maximize its performance me asure - which is neither omniscience nor perfection, and is bounded by its own resources
- PEAS writes down the four things rationality depends on. Always step one
- Seven properties classify any environment, and each one makes a demand on the agent
- Four architectures, each fixing what the previous could not:
  - reflex → model-based → goal-based → utility-based
  - Learning cuts across all four

Where This Comes Back

- Almost everything ahead is a return to today's vocabulary
- Weeks 3 to 5, search
  - How a goal-based agent actually finds its sequence of actions, using the transition model from today
- Weeks 6 and 7, constraints and games
  - What changes when there are rules, or an opponent
- Further out, decision processes and reinforcement learning
  - The expected-discounted-utility line, taken seriously

Next Lecture: Solving Problems by Searching

- A goal-based agent needs a sequence of actions, not just one
- Finding that sequence is search
- What a search problem is, stated precisely
  - States, actions, transitions, a start state, and a goal test
- State spaces and search trees - and why they are not the same thing
- Breadth-first and depth-first search, and what each one costs
- Read: Chapter 3 (3.1–3.4, pp. 81–101)

Introduction to Artificial Intelligence

Chapter 2 - Part III: Agent types

Namyup Kim

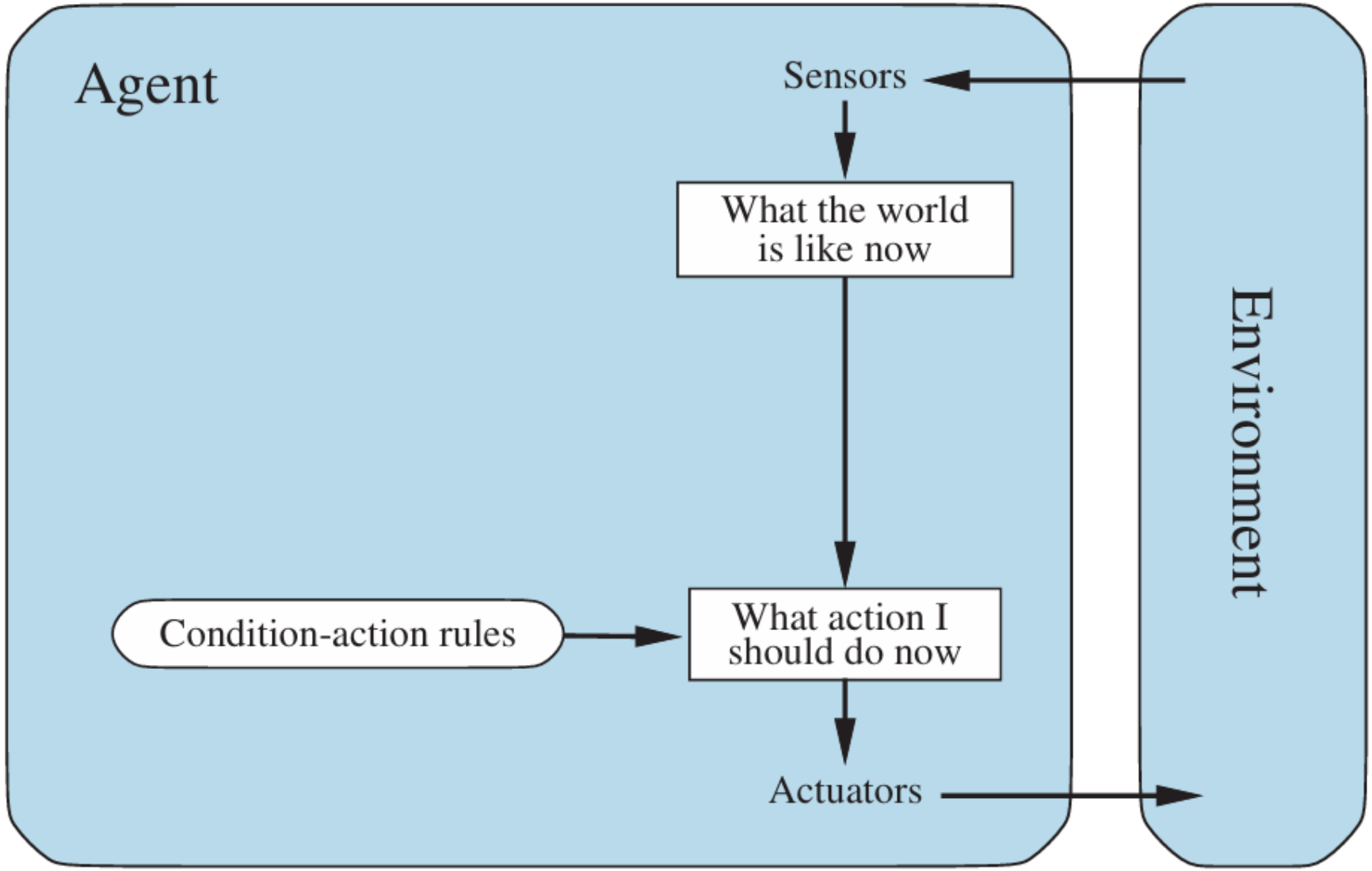
Soongsil University



A Hierarchy of Agent Types



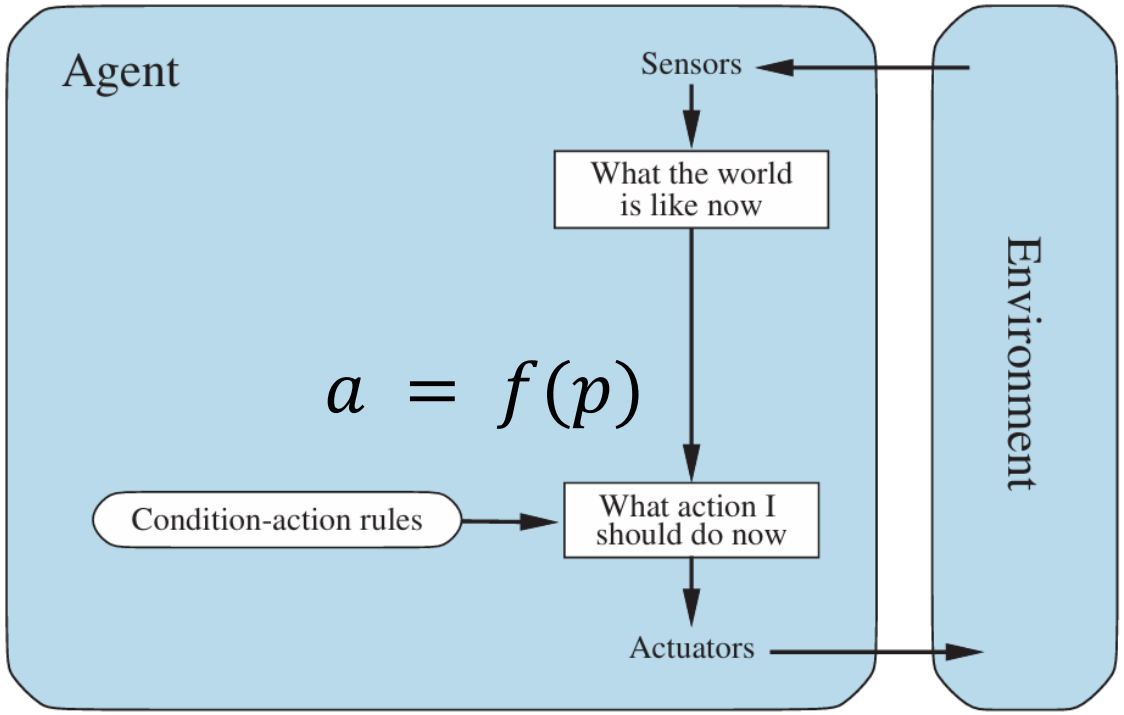
Simple Reflex Agent



Acts on the current percept alone.

How a Reflex Agent Runs

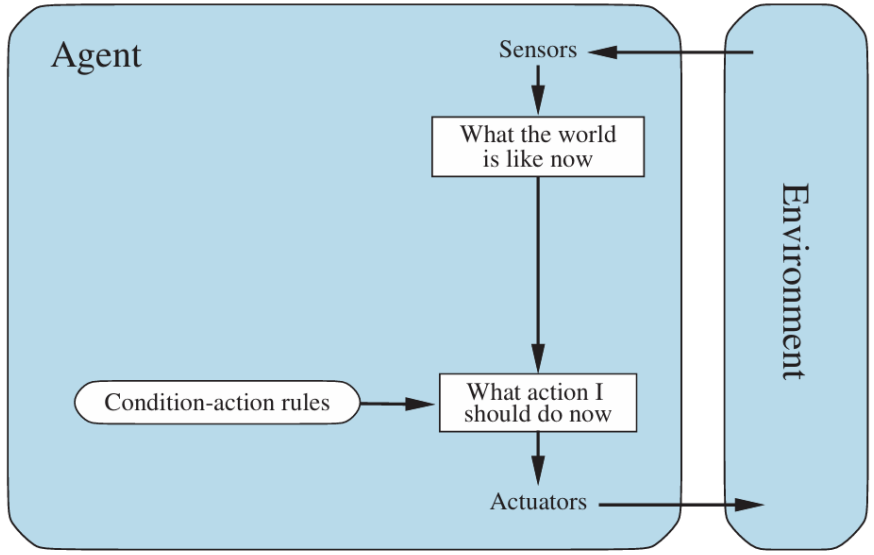
- The agent function is the current percept alone: $a = f(p)$
- So, the interaction is a simple alternation: $p_0, a_0, p_1, a_1, p_2, a_2, \ldots p_t, a_t$
- Fast, and cheap. It also does not know why it does anything
  - It does not know the performance measure at all - yet well-chosen rules can still p erform well



Example of Simple Reflex Agent

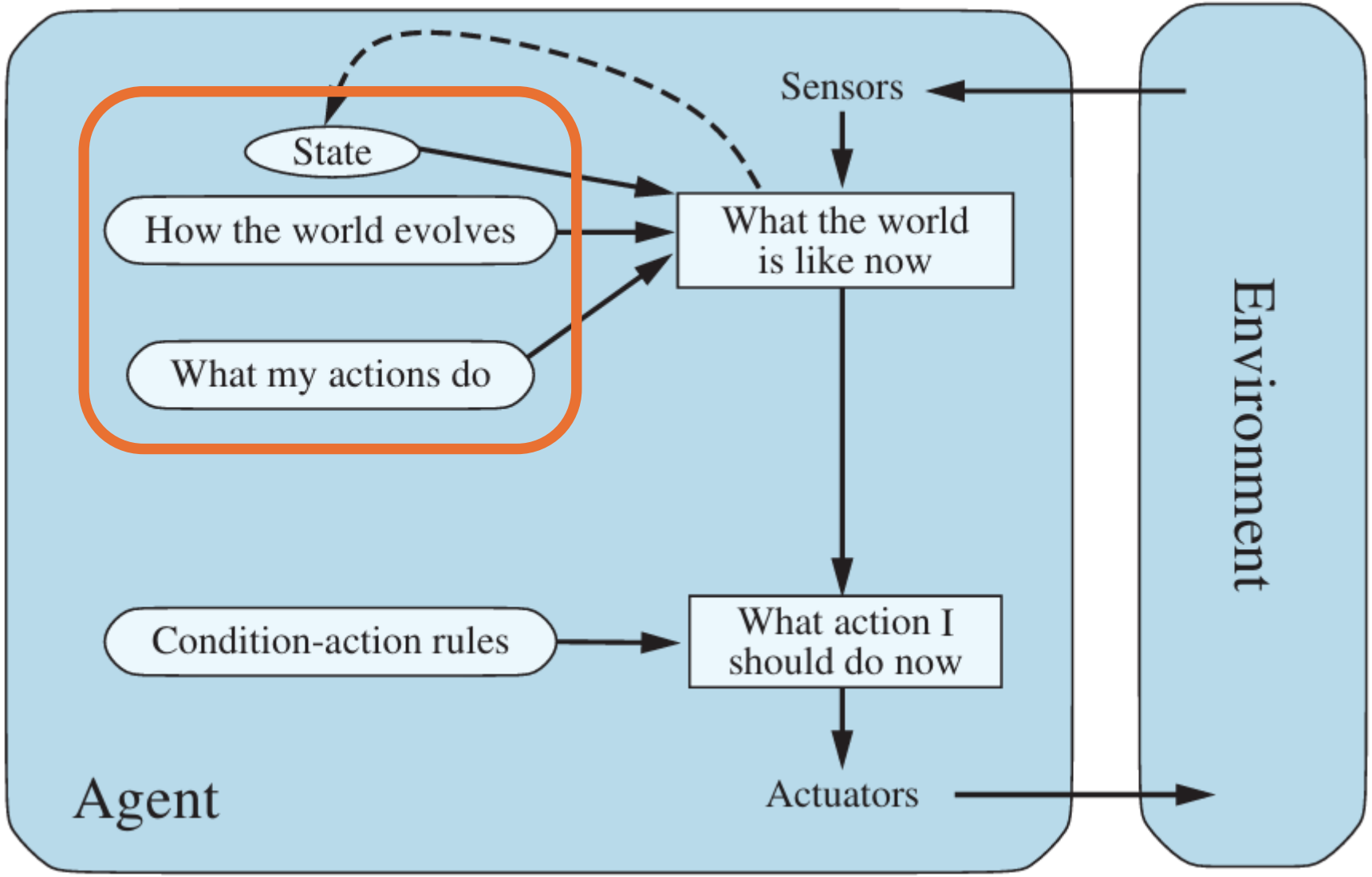
- Rule-based reflex vacuum cleaner:

```
function REFLEX-VACUUM-AGENT([location,status]) returns an action
    if status = Dirty then return Suck
    else if location = A then return Right
    else if location = B then return Left
```



- Driving: If the car in front is braking, brake. Everything before this instant (history) is ignored.

Model-Based Reflex Agent

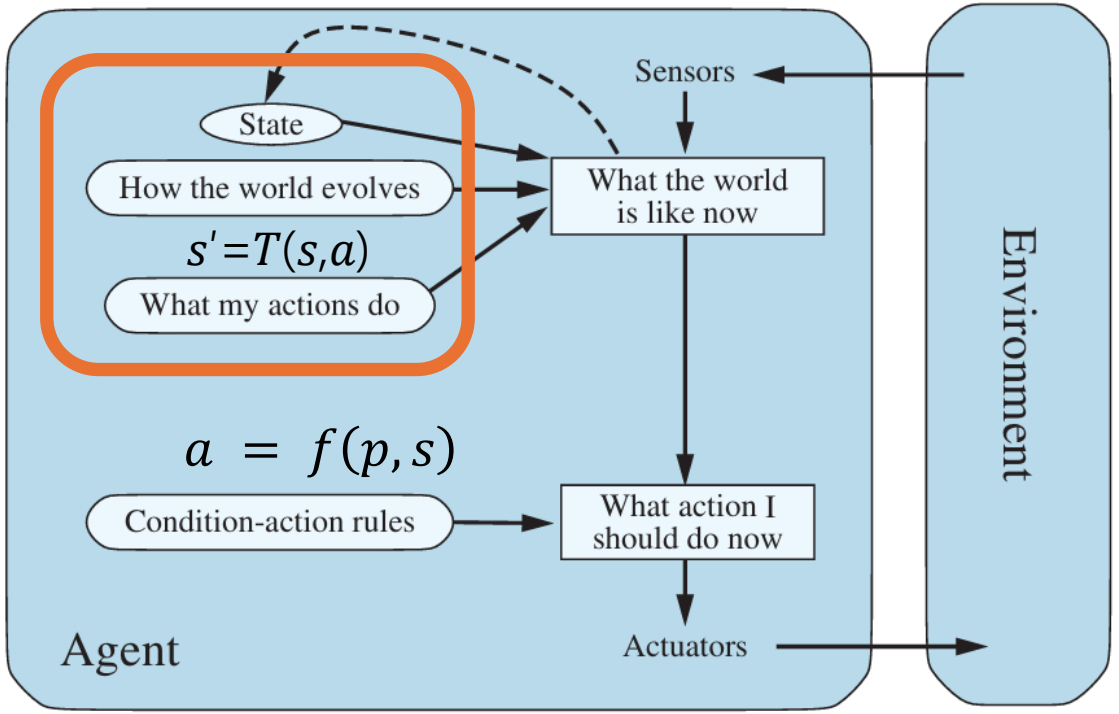


Model-Based Reflex Agent: Adding Memory

- The agent function now takes a second argument
  : $a = f(p, s)$ where s is state, updated at every step.

- It can now track what it cannot currently see
  - Memorize a car in the next lane, while you are looking the other way
  - Memorize which rooms already cleaned

- More information available means better rules become possible

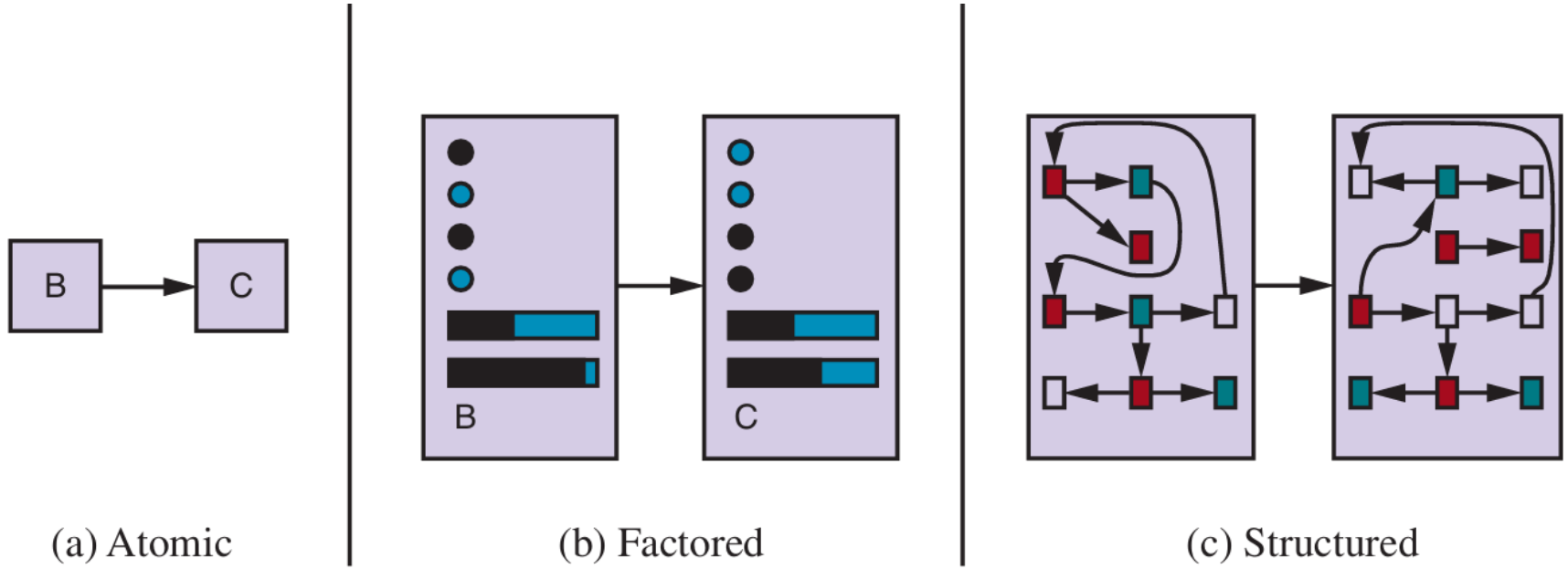


Model-Based Reflex Agent: Two Models Are Needed

- To maintain that **state variable**, the agent needs two kinds of knowledge
- A **transition model**:
  - How the world changes - both from its own actions a and on its own world
- A **sensor model**:
  - How the state of the world shows up in its percepts
- One honest caution: the state is a best guess in by sensor, never a certainty
  - In a partially observable world the exact state is rarely knowable

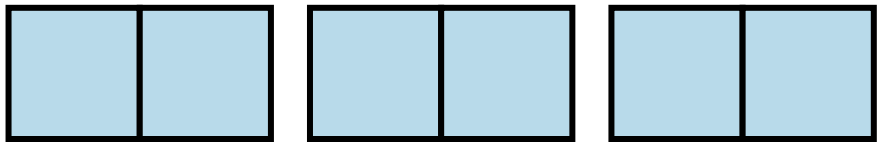
Model-Based Reflex Agent: State Representation

- **State** helps the agent keep track of the environment
  - Atomic - the state is a label.
    - Just “A” or “B”.
  - Factored - the state is a set of attribute-value pairs, called fluent
    - [location = A, status = clean, temperature = 24°C]
  - Structured - objects, with attributes and relations between them
    - “the cup is on the table, which is in the kitchen”



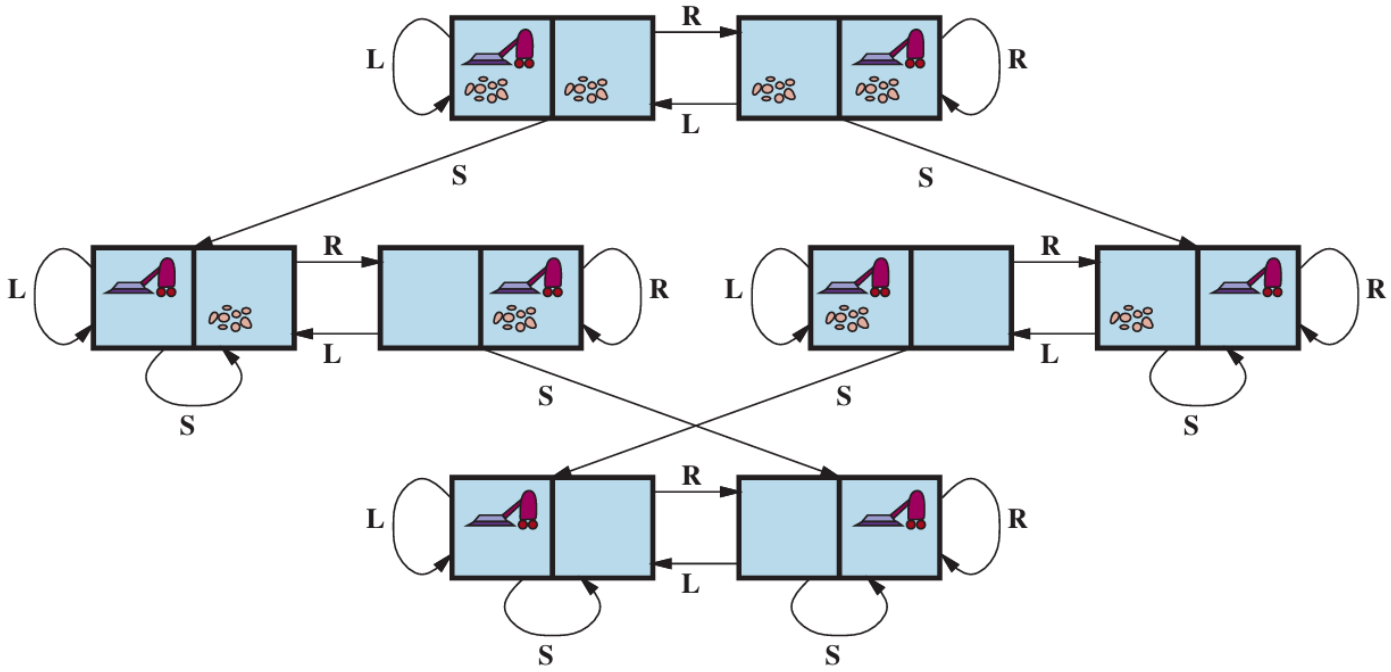
Model-Based Reflex Agent: The Transition Model

- Treat the environment as a discrete dynamic system
- The transition function says where an action takes you:
  - the set of states:



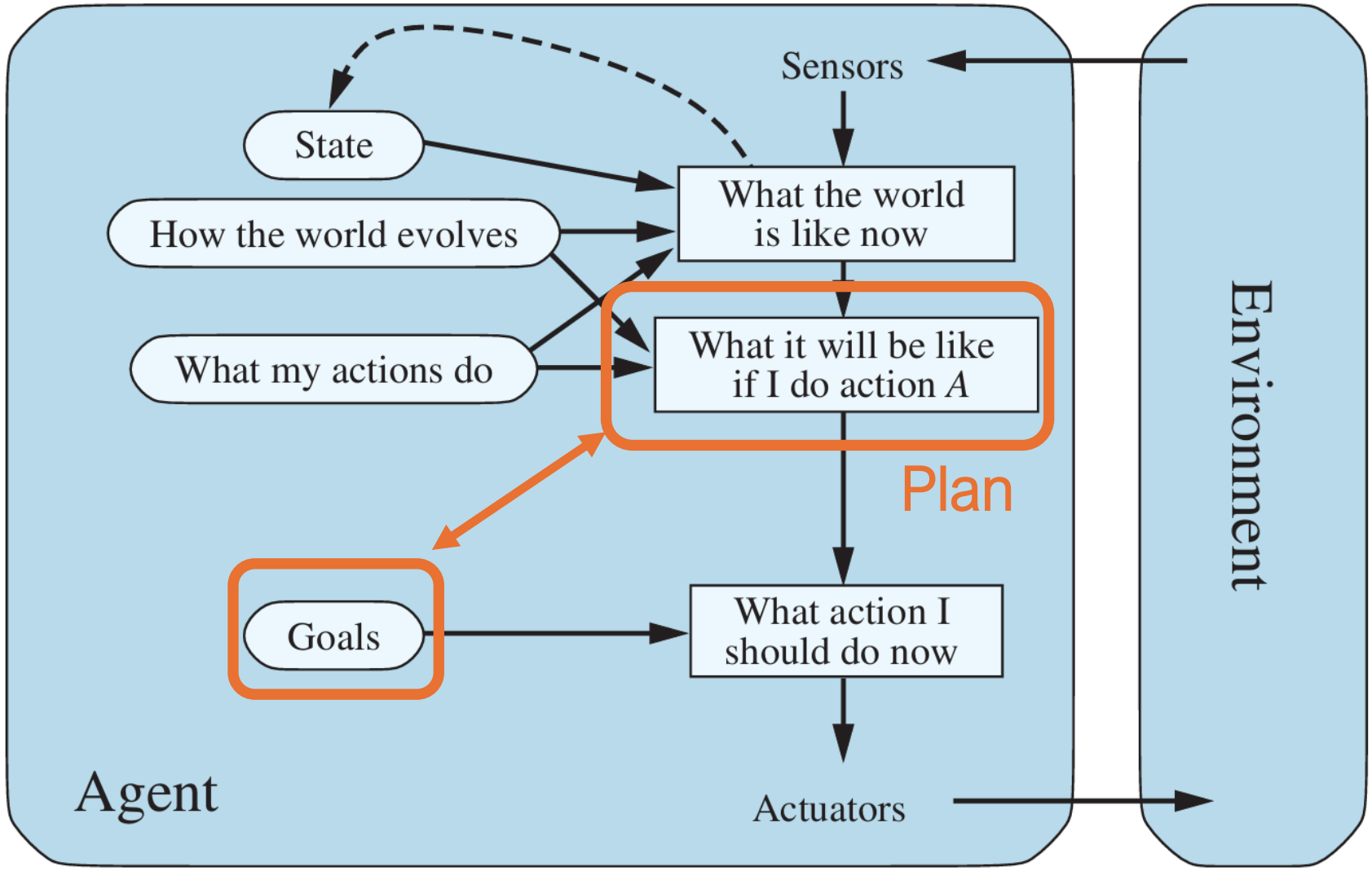
  - A the set of actions:





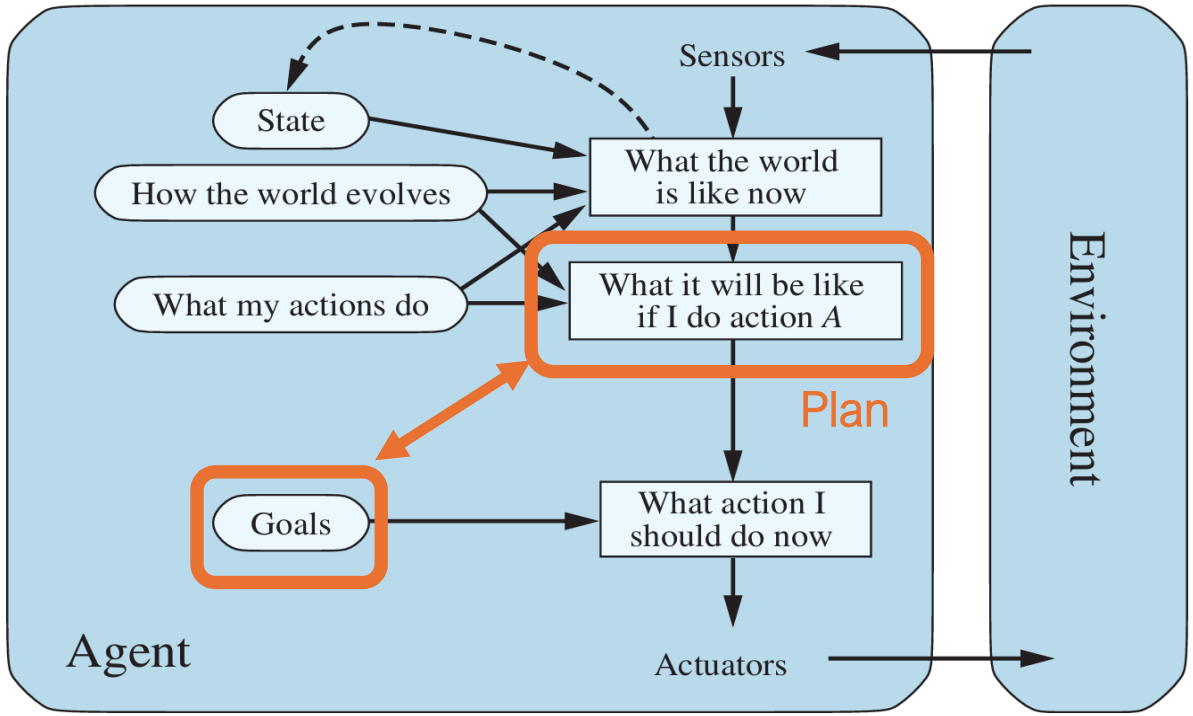
- *Next week this becomes the backbone of every search algorithm*

Goal-Based Agent



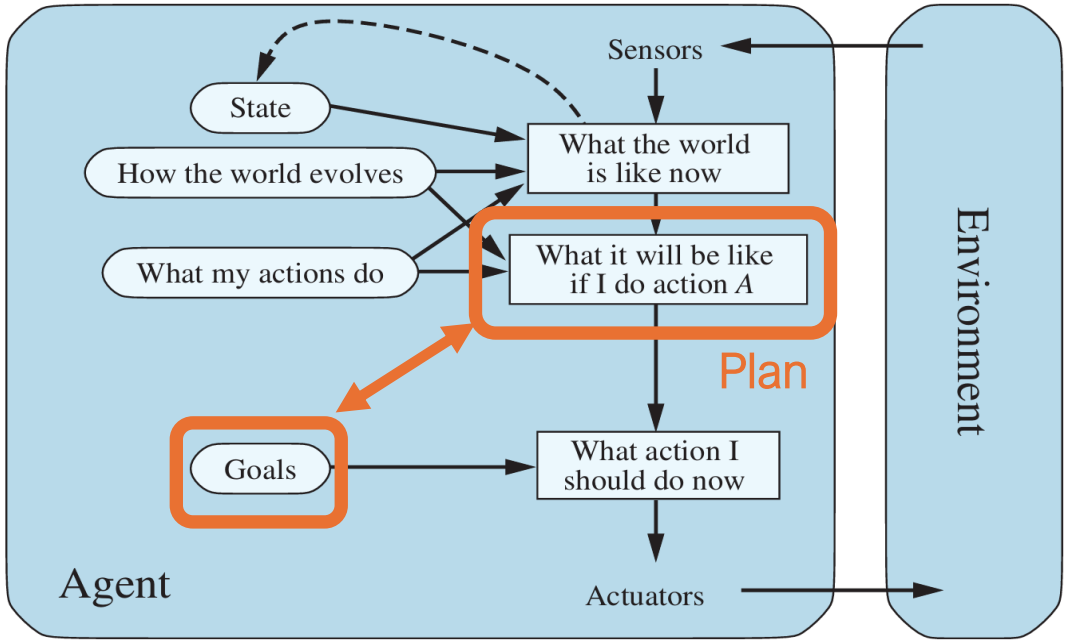
Goal-Based Agent: Flexibility, Concretely

- Has a goal state.
  : The agent is given a defined goal state to reach, and the task **ends** once it gets there.
- Must act toward the goal state.
  : **A planning** uses **search algorithms** to work out a sequence of actions that leads to the goal.

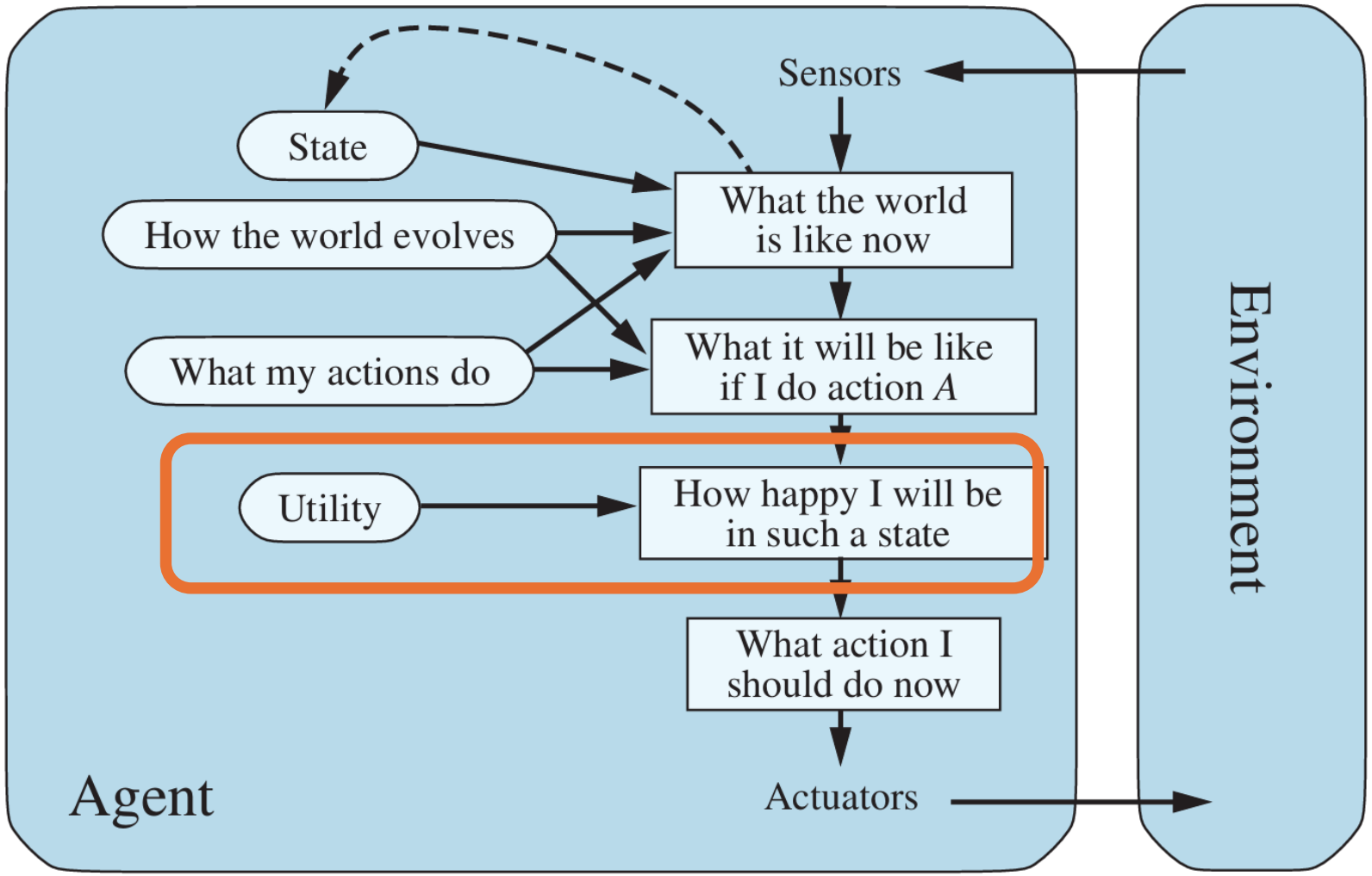


Goal-Based Agent: Examples

- Send a goal-based taxi somewhere new
  - Change one thing: the destination
  - The plan is changed for the destination
- Send a reflex taxi somewhere new
  - Every rules for when to turn and when to go straight will work only for a single destination.
  - Replace all of them
- The difference is that ***the goal agent's knowledge is written down explicitly***, and so can be changed.



Utility-Based Agent



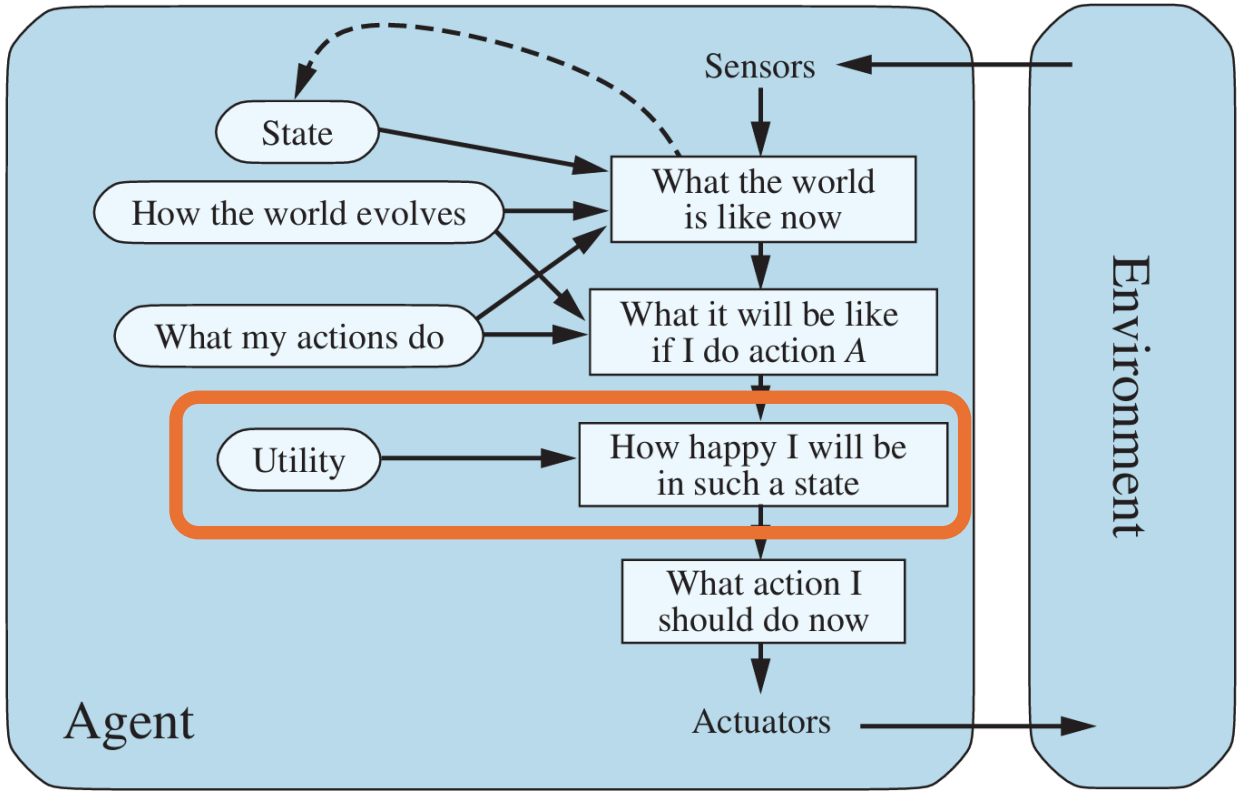
A goal is a yes-or-no test. A utility function scores how good a state is, which is what you need when several routes reach the goal.

Utility-Based Agent: When Goals Are Not Enough

- Three situations a goal test and a path cost cannot handle
  - There is no terminal goal at all
    - A Mars rover keeping its battery out of the danger zone. Nothing to arrive at. The task never ends.
  - Objectives conflict
    - Fast against safe against comfortable. You cannot have all three, and someone has to write down the trade-off.
  - Outcomes are uncertain
    - 90% chance of a small win, 10% chance of a large one. A yes-or-no test cannot multiply by a prob ability

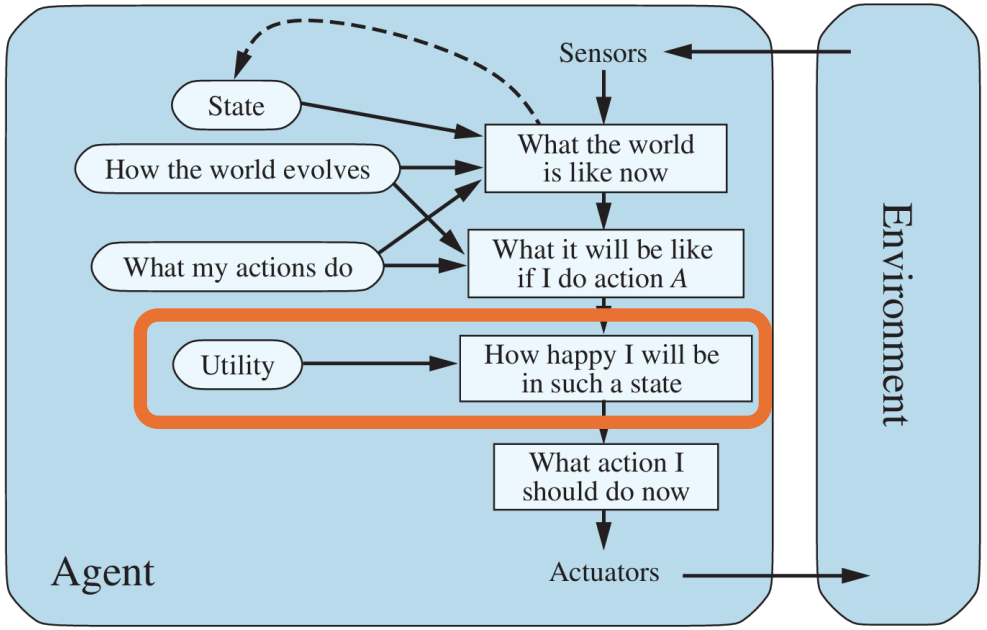
Utility-Based Agent: When Goals Are Not Enough

- Goals just provide a binary distinction between “happy” and “unhappy” states.
- A utility function scores how desirable (happy) a state is
  - Often written as a reward $\boldsymbol{R(s)}$ attached to each state.



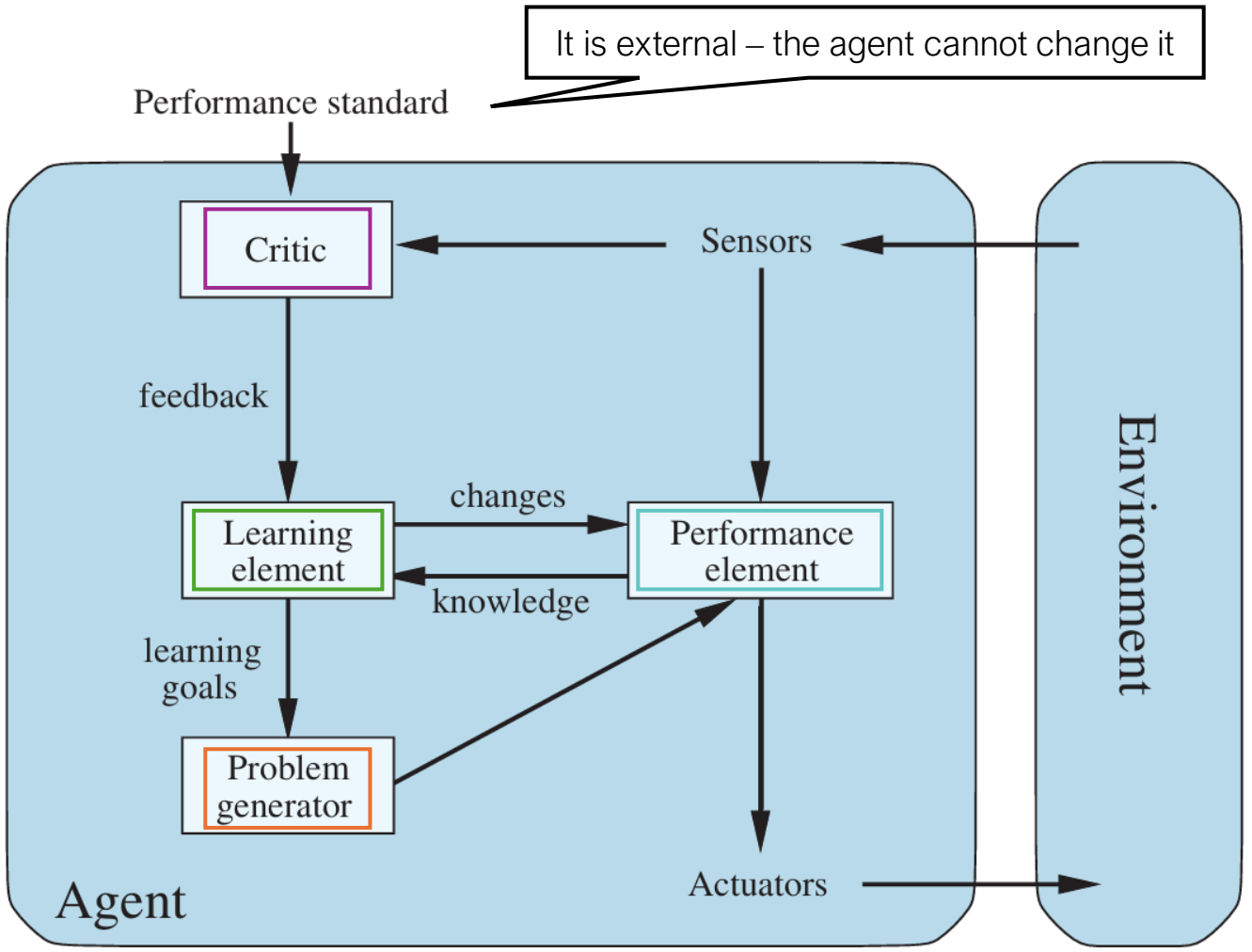
Goal-Based Agent vs Utility-Based Agent

- Goal-based
  - $p_0, s_0, a_0, p_1, s_1, a_1, \ldots, s_{\text{goal}}$
  - It terminates. The agent arrives, and it is done
- Utility-based
  - $p_0, s_0, a_0, p_1, s_1, a_1, \ldots$
  - It does not terminate. There is no arriving, only continuing



Learning Agent

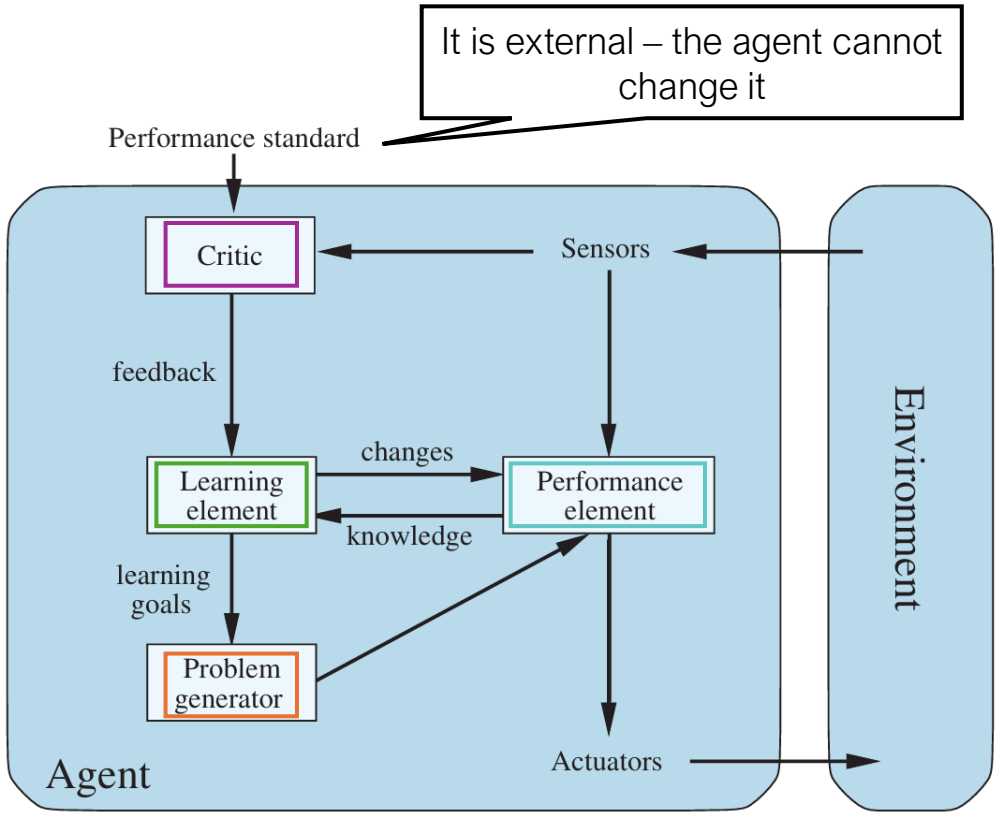
- Performance element
  - It takes percepts and chooses actions. That is the entire agent.
- Learning element
  - Updates the performance element so it does better next time
- Critic
  - Judges what happened against a fixed external standard
- Problem generator
  - Suggests intentionally actions worth trying



Learning Agent: Two Things Worth Noticing

- The performance standard must sit outside the agent
  - An agent able to edit its own standard would lower it to whatever it already achieves.
  - The same failure as a gameable performance measure in the vacuum world.

- Learning is orthogonal to everything else in this section
  - Any of the four architectures can be built to learn, or not



Utility-based agents

Goal-based agents

Model-based reflex agents

Simple reflex agents

Can be built as learning agent

Summary

- An agent perceives through sensors and acts through actuators; the agent function maps percept sequences to actions
- A rational agent takes the action expected to maximize its performance me asure - which is neither omniscience nor perfection, and is bounded by its own resources
- PEAS writes down the four things rationality depends on. Always step one
- Seven properties classify any environment, and each one makes a demand on the agent
- Four architectures, each fixing what the previous could not:
  - reflex → model-based → goal-based → utility-based
  - Learning cuts across all four (learning agnet)

Where This Comes Back

- Almost everything ahead is a return to today's lecture
- Weeks 3 to 5, search
  - How a goal-based agent actually finds its sequence of actions, using the transition model from today
- Weeks 6 and 7, constraints and games
  - What changes when there are rules, or an opponent
- Further out, decision processes and reinforcement learning
  - The expected-discounted-utility line, taken seriously

Next Lecture: Solving Problems by Searching

- A goal-based agent needs a sequence of actions, not just one
- Finding that sequence is search
- What a search problem is, stated precisely
  - States, actions, transitions, a start state, and a goal test
- State spaces and search trees - and why they are not the same thing
- Breadth-first and depth-first search, and what each one costs
- Read: Chapter 3 (3.1–3.4, pp. 81–101)